UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Escola Tècnica Superior d'Enginyeria
de Telecomunicació de Barcelona

Work-flux optimization for the use of GRASP algorithm in quasi-real-time

Master Thesis
submitted to the Faculty of the
Escola Tècnica d'Enginyeria de Telecomunicació de Barcelona
Universitat Politècnica de Catalunya
by

Mireia López Barrón

In partial fulfillment
of the requirements for the master in
(TELECOMMUNICATIONS) **ENGINEERING**

Advisor: Alejandro Rodriguez Gomez
Barcelona, Date XXXXX

Contents

List of Figures

List of Tables

Revision history and approval record

Revision	Date	Purpose
0	10/12/2025	Document creation
1	13/01/2026	Document revision
2	23/04/2026	Document revision

DOCUMENT DISTRIBUTION LIST

Name	e-mail
Mireia López Barrón	mireia.lopez.barron@estudiantat.upc.edu
Alejandro Rodriguez Gomez	alejandro.rodriguez.gomez@upc.edu

Written by:		Reviewed and approved by:	
Date	dd/mm/yyyy	Date	dd/mm/yyyy
Name	Mireia López Barrón	Name	Alejandro Rodriguez Gomez
Position	Project Author	Position	Project Supervisor

Abstract

The need to monitor atmospheric aerosols has grown significantly due to their impact on health and climatology. The Generalized Retrieval of Atmosphere and Surface Properties (GRASP) is a unified algorithm for obtaining these properties from remote sensing observations. This thesis presents the development of a software application that automates the full GRASP execution workflow in quasi-real-time. Built using C# and the Avalonia UI framework, the cross-platform application seamlessly integrates sequential data processing steps. It automatically downloads photometer data from AERONET and Lidar data from ACTRIS-EARLINET, combines them using Matlab scripts, executes the GRASP algorithm, and visualizes the results. The final product is a reliable, user-friendly tool, fully compatible with Linux environments like the CALCULA system. By automating this complex workflow and incorporating robust error handling, the application significantly reduces manual effort and accelerates atmospheric data analysis.

1 Introduction

Recently, the need to analyse and monitor atmospheric aerosols has grown significantly, as their negative impact has grown significantly in different sectors: human health, climatology and air transport. GRASP (Generalized Retrieval of Atmosphere and Surface Properties), which was introduced by Dubovik, is the first unified algorithm used for obtaining atmospheric properties from different remote sensing observations including satellite, ground-based and airborne measurements, both passive and active, of atmospheric radiation and their combinations[1].

The main objective of this work is to develop a software that automates the full GRASP execution process, which includes:

1. Download photometer data from the Aeronet website
2. Download Lidar (Light Detection and Ranging) data from the ACTRIS-EARLINET website
3. Combine the data downloaded from both websites
4. Run GRASP
5. Plot the obtained results

1.1 Gantt Diagram

The time organization of the project is shown in the Gantt diagram of Figure 1 representing a time lapse of one semester. It is important to note that even though there are different groups of tasks for implementation and testing, tests have been executed during the implementation phase to ensure the correct functionality of the different steps.

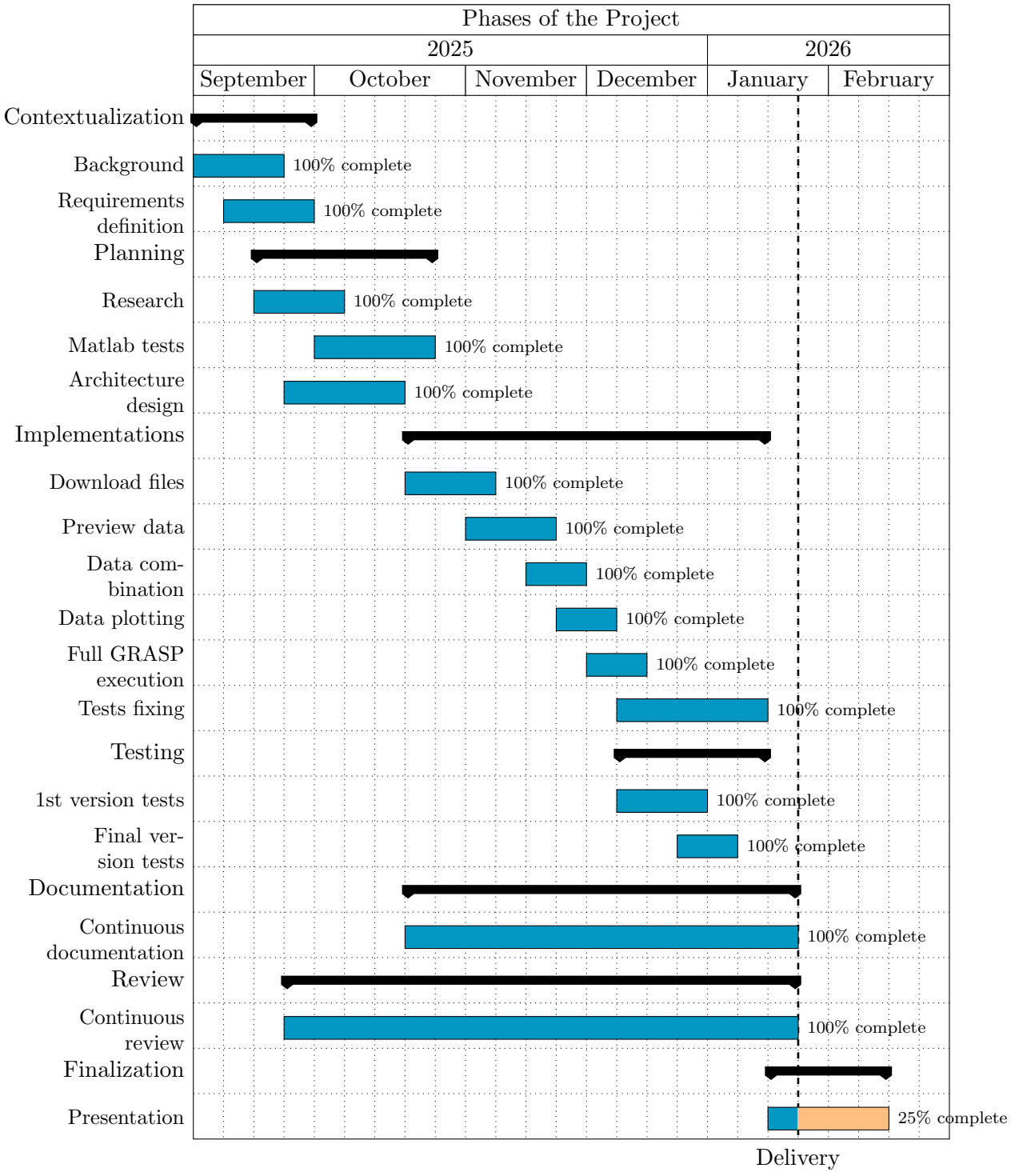


Figure 1: Gantt diagram of the project

1.2 Deviation from the initial plan

Different issues were encountered during the development of the application, which led to deviations from the initial plan and the initial timeline. The main problems were:

- Not precise project requirements definitions: The initial requirements were not precise enough, which led to a lot of changes during the development of the application.
- Not enough knowledge of testing scenarios: During the first phase of testing, it was discovered a larger list of testing scenarios, since it was supposed that the combination of files downloaded and it's content were not as diverse as it were in reality.
- Difficult understanding of previous code: The previous code was kaotic and difficult to understand during the inital phase of the project, even if there was documentation. In addition, some code was obsolete due to the modifications to files formats.
- No demo available of previous work: There was no demo available of previous work, which made it difficult to understand the application main objective, desired behaviour and functionality.

2 State of the art

2.1 GRASP algorithm

GRASP (Generalized Retrieval of Aerosol and Surface Properties) is a unified and highly accurate algorithm used to study the atmosphere. It retrieves many different aerosol and surface characteristics from remote-sensing observations. According to the GRASP-OPEN website, the algorithm can estimate **nearly 50 parameters**, such as particle-size distribution, refractive index, sphericity, and absorption. It is designed to work with **spectral, multi-angle, and polarimetric** measurements, and it performs well even over **bright surfaces** like deserts, where the ground reflectance usually hides the aerosol signal.

It combines information from different instruments—both satellite and ground-based—to produce a detailed description of aerosols and surface reflectance. Its main features include:

- Processing reflectance data from aerosols and land surfaces to infer physical and optical properties.
- Using multi-angle and polarimetric observations, which provide more information than simple intensity measurements.
- Integrating diverse data sources, making it adaptable to many observation systems.
- Applying a unified inversion approach, meaning it uses the same mathematical framework for all types of input data. This helps ensure consistency and accuracy across different instruments [1].

This algorithm is valuable because it offers a comprehensive and consistent way to analyse atmospheric aerosols. These particles affect climate, air quality, and visibility, so understanding them is essential. GRASP's ability to work with many types of measurements makes it suitable for:

- Climate and environmental studies
- Air-quality monitoring
- Satellite-data validation
- Research on aerosol–surface interactions

2.1.1 Input data

GRASP algorithm aims to analyse different types of measurements in order to The UPC lidar data used in GRASP come from the Single Calculus Chain (SCC), an online system for processing and storing lidar measurements. The lidar information includes:

- Elastic backscatter at 355, 532, and 1064 nm: these channels record the total backscattered signal at each wavelength. Although the laser emits linearly polarized light, these channels do not separate polarization. The signals are taken from the SCC's ELPP module and pre-processed by:

  - applying distance correction,
  - resampling to 60 logarithmic points,
  - normalising the profile.

  This data is mandatory in order to run the GRASP algorithm.

- Depolarized backscatter at 355 and 532 nm: these channels detect only the component of the signal perpendicular to the transmitted polarization. GRASP cannot use these signals directly, so the volume depolarization must be calculated. This is done in the SCC's ELDA module.

The application developed in this project automatically checks which lidar measurements are available for each date and identifies whether total-power and/or depolarization data exist at the required wavelengths.

2.1.1.1 Photometer data

The CIMEL CE318 photometer collects different types of measurements, organized in "scenarios," which are automatically sent to the AERONET server and later used as input for GRASP. The main scenarios include:

- Aerosol Optical Depth (AOD): a direct measurement from sunlight or moonlight, available both during the day and on moonlit nights. The result is adjusted to represent a vertical atmospheric column. This data is mandatory in order to run the GRASP algorithm.
- Almucantar scans: the instrument measures diffuse irradiance while rotating in azimuth at a fixed solar elevation. This data is mandatory in order to run the GRASP algorithm.
- Polarized almucantar scans: similar to the previous case, but measuring the degree of linear polarization (DoLP) introduced by aerosols.
- Main-plane scans: measurements taken by sweeping in elevation while keeping the solar azimuth fixed, for both total irradiance and DoLP.
- Hybrid scenarios: combinations of the above.

GRASP can currently combine AOD, total-irradiance almucantar data, DoLP, and lidar measurements. Only a limited number of photometers worldwide provide polarized almucantar data, so the system must check whether these measurements are available for each date.

During calibration periods, the replacement photometer may not record DoLP, which affects the available inputs.

2.2 Current execution flow

Universitat Politècnica de Catalunya (UPC) research department for Remote Sensing has been using the GRASP algorithm to process and obtain information from data measured

by a lidar and a photometer, in order to study the presence of aerosols in the atmosphere. In order to do that, different steps need to be executed manually, without a graphical interface or centralized application, which increases the risk of errors, reduces reproducibility, and demands significant user intervention.

Other students have previously developed Matlab scripts in order to automate the preprocessing stage of the workflow used by the department. However, this solution is not optimal as it requires the user to have a basic knowledge of Matlab and all the different web sites and files used.

Every time someone wants to use the GRASP algorithm, they need to follow the steps described in the different points of this section.

2.2.1 Download data

2.2.1.1 Earlinet

First website to download data is Actris-Earlinet. This interface is used to download the data measured by the Lidar, and can be easily downloaded by configuring different parameters like start and finish date, and selecting the desired station, as it can be seen in Figure 2. The different files that can be downloaded will appear listed under the configuration parameters list.

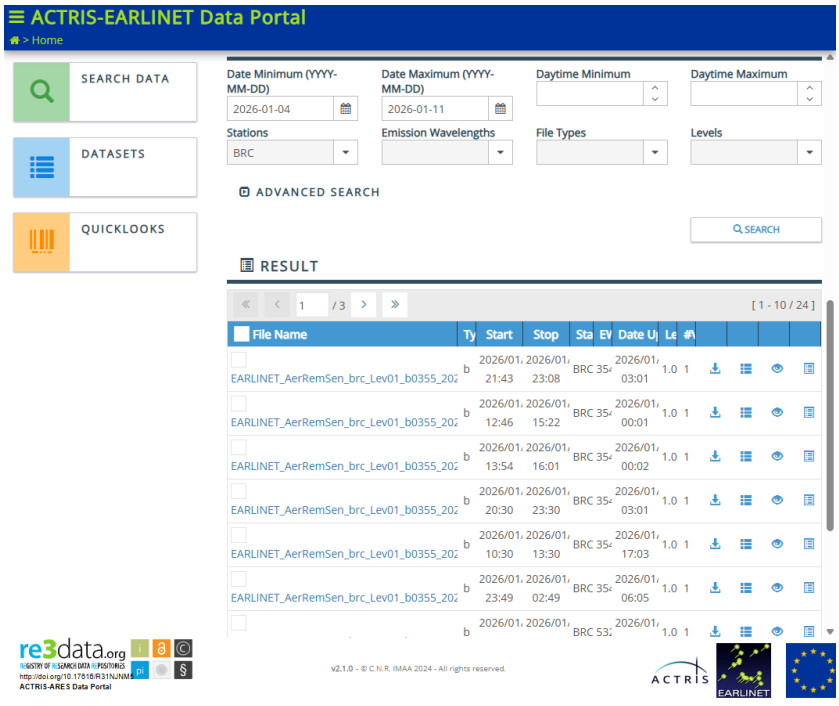


Figure 2: Earlinet website for downloading data

From this website, it is always downloaded Range Cross Signal in wavelengths 355, 532 and 1064 nm. There are exception periods, where 1064 nm signal is not downloaded. In this case, GRASP cannot be executed.

Volume Depolarization can be also downloaded, but it is not always present in the downloaded measurements in some periods of time. Even if this data is missing, GRASP can still be executed without problem by using the correct configuration.

2.2.1.2 Aeronet

Aeronet website offers more information and options to download data, so it needs to be done in different steps. First, the user needs to locate the Aeronet Data Access page, as it can be seen in Figure 3, and select a data option.

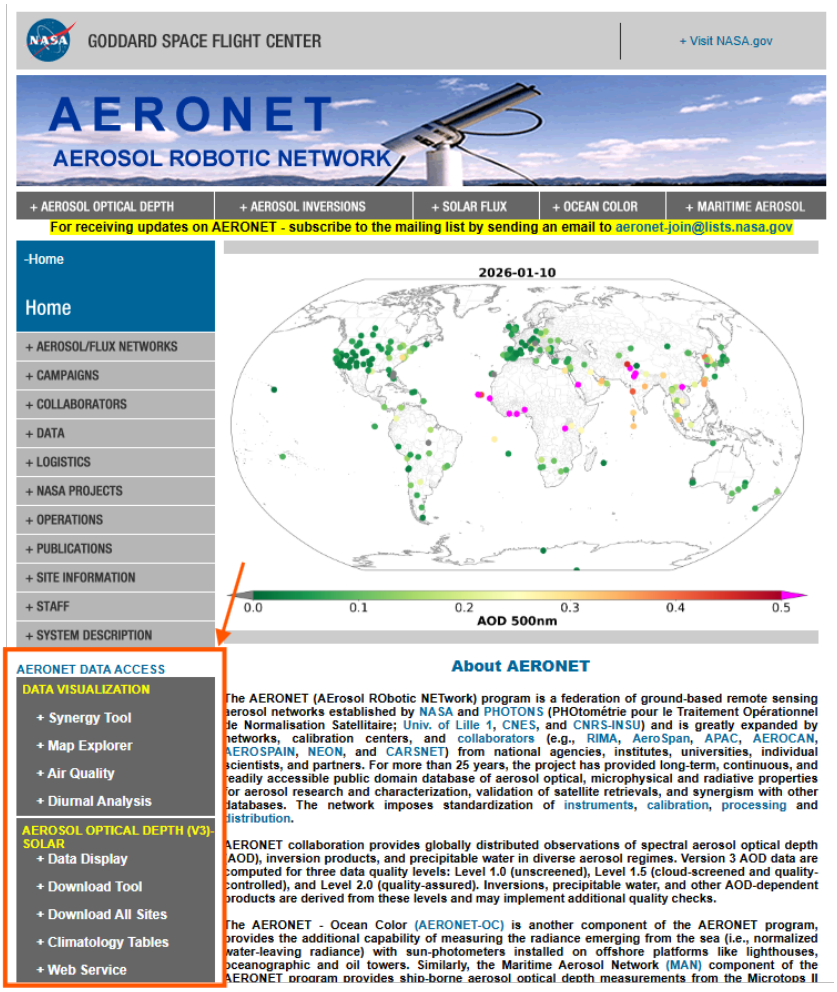


Figure 3: Aeronet website for downloading data

For each type of data, the user needs to select the desired site from the shown list, as it can be seen in Figure 4.

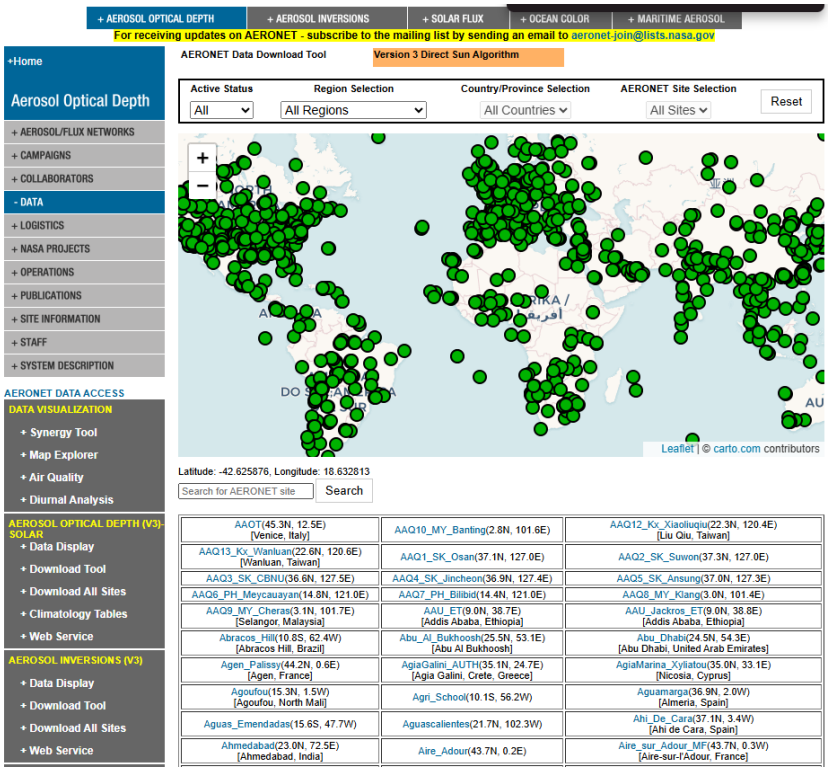


Figure 4: Aeronet site selection

Once the site is selected, the user needs to choose the different data options to be downloaded for both, AOD and Inversions, as it can be seen in Figure 5 and Figure 6, respectively.

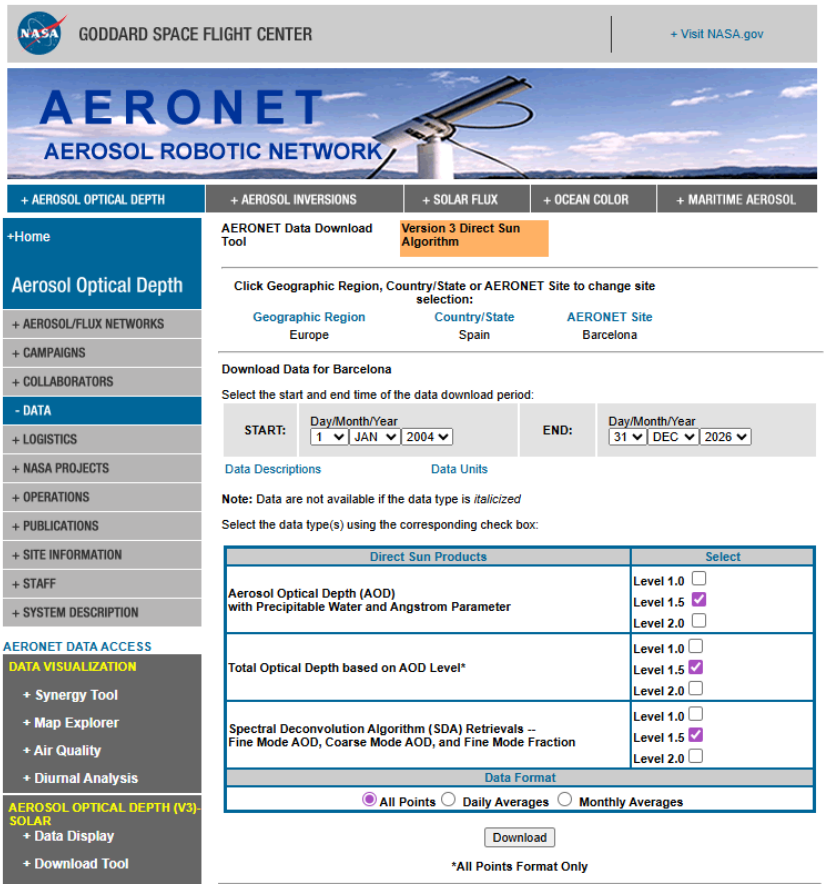


Figure 5: Aerosol Optical Depth download UI

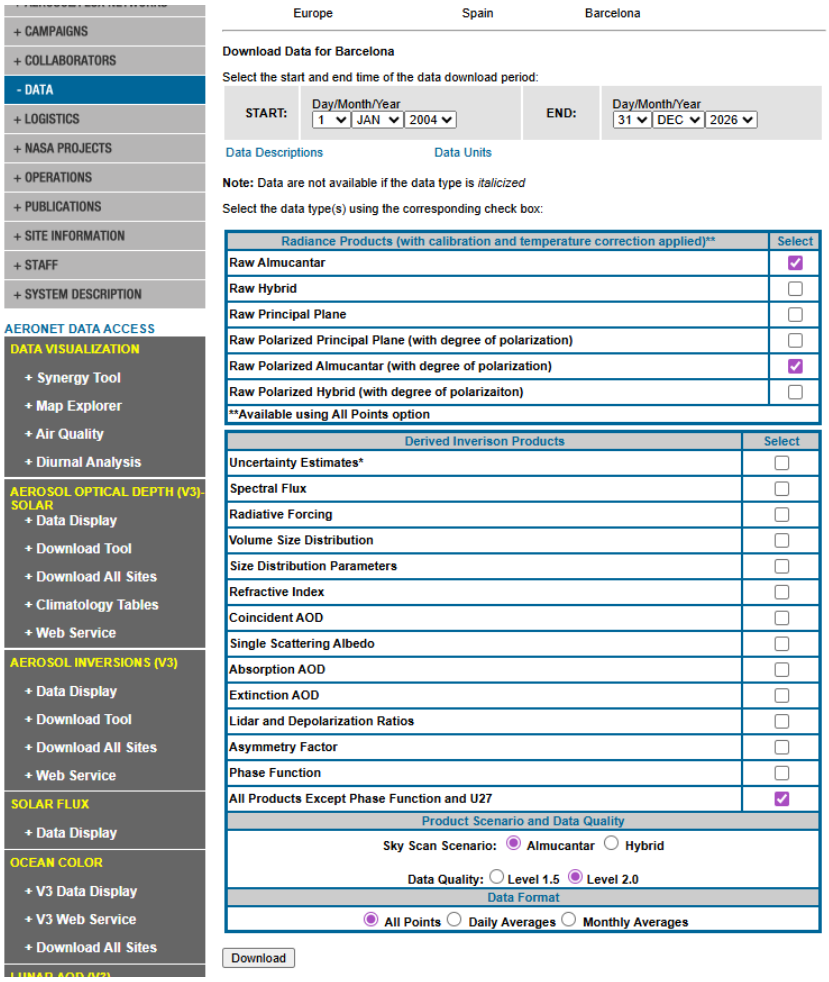


Figure 6: Aerosol Inversions download UI

It can also be downloaded the inversions results from the AERONET website **??**, which it is used for comparing the results after executing the GRASP algorithm.

Once all data has been downloaded, it needs to be located in a file accessible for the Matlab scripts.

2.2.2 Select configuration and execute Matlab scripts

There are different configurations, depending on the available data downloaded:

- D1L: Only photometer and Lidar data are available
- D1P_L: It is also available polarization data in photometer measurements
- D1L_VD: It is also available depolarization data in Lidar measurements
- D1P_L_VD: All data is available

Knowing the available files and the selected dates, in order to run the Matlab Scripts successfully, the user should manually choose the configuration and execute the corresponding

script, since there is one script per configuration possibility.

2.2.3 Modify manually configuration file for GRASP algorithm

After executing the Matlab scripts, a data file is created named ¡measureID¿_GARRLIC_¡configuration¿.s which will be used as input for the GRASP algorithm. But a configuration file is also needed, that indicates different parameters to the algorithm, being one of them the name of the data file to be used. This file is named UPC_¡configuration¿.yml.

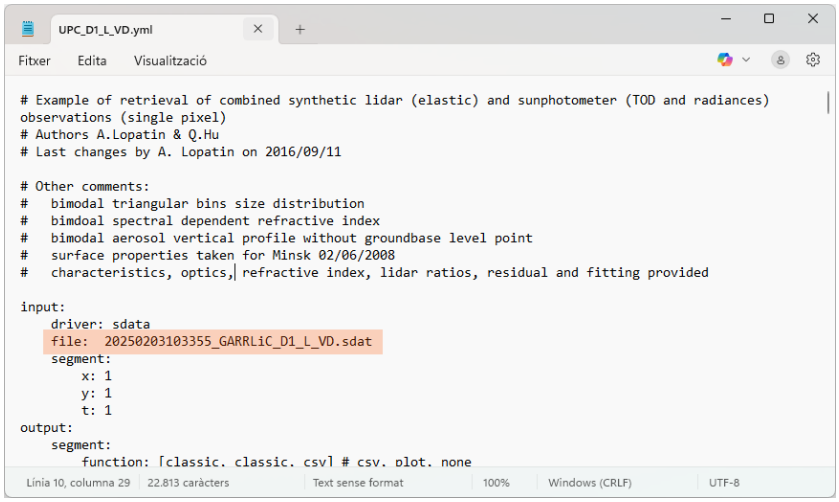


Figure 7: Example of GRASP configuration file

In Figure 7, it can be seen the example of a configuration file when the configuration chosen is D1L_VD and the MeasureID is 20250203103355, corresponding to the 3rd of February at 10:33:55.

2.2.4 Execute GRASP algorithm in CALCULA

When all required files have been created and correctly configured, the GRASP algorithm can be executed in CALCULA. The user should upload the files to the virtual environment and execute the following command lines in the command prompt:

```
cd <directory to files>
/opt/bin/grasp-1.1.5/grasp UPC<config>.yml
```

Once the algorithm has been executed correctly, files named UPC_¡configuration¿_screen.txt and UPC_¡configuration¿_out.txt will be created in same directory.

2.2.5 Execute Matlab script for postprocessing and results analysis

After the execution, the user needs to download the output files from CALCULA and execute the Matlab post processing script in order to obtain the results and the corresponding figures to analyse.

2.2.6 Need of optimisation

After analysing the process, it can be seen that it is not optimal as it requires the user to access different websites in order to download the needed files, analyse the available data in order to select the correct configuration and execute scripts manually, without a graphical interface or centralised application, which increases the risk of errors, reduces reproducibility, and demands significant user intervention.

Working with different Operating Systems (mostly Windows and CALCULA) also increases the time needed to execute the process.

The main objective of this project is to optimise all of the steps of this sequence so the time consumption and the error introduction possibilities by the user are reduced.

3 Methodology

In order to create a new application, it is necessary to follow the following steps, to create a strong foundation.

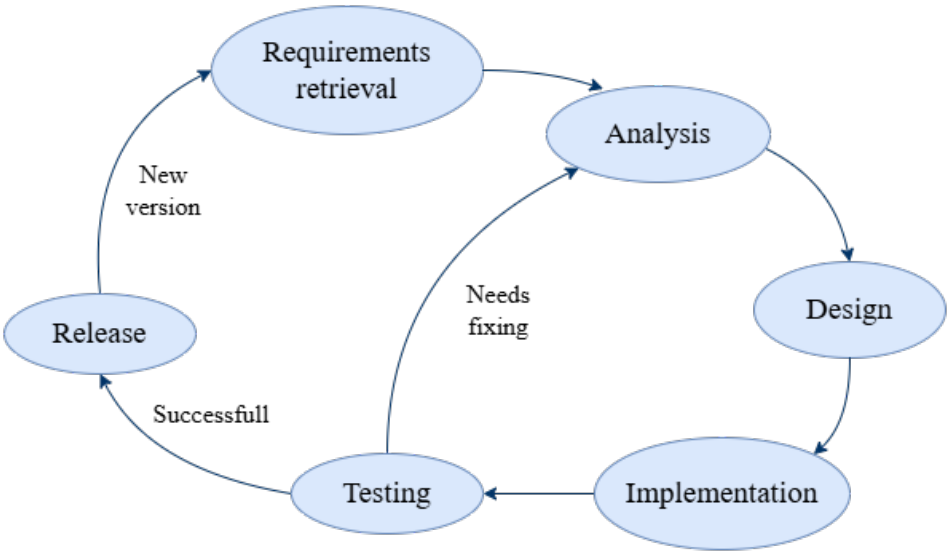


Figure 8: Project development cycle

This diagram shows the ideal development process for a project. Since the application developed is the first version, requirements retrieval should only happen once, and the project should finish with the Release phase, once all tests have been passed correctly.

For the development of the application, it has been decided to use a hybrid structure, using Object Oriented Programming (OOP) with C# and Matlab scripting.

The main advantages of using C# with OOP are that it allows the code to be reusable, organised and easy to maintain [2]. In this project, it has been used for the development of the Graphical User Interface and the controllers for both, web services and Matlab script management.

On the other hand, Matlab provides a specialised environment for numerical calculations, simulations, and complex data analysis[3]. Working directly with matrices and arrays makes the data processing and visualisation easier and more efficient.

Keeping all the Matlab-scripts execution inside the application will also help the analyisis of the results of the corresponding plots by keeping the enviroment and tools offered by Matlab, which are the ones used in the present.

Having the Matlab code in the execution directory also offers the possibility to update the different scripts in case it is needed in the future. One exemple of this would be the modification of the different file formats or the desire to add a new parameter in any plot. For the final user, which is the UPC research group, not knowing how the application works would not be a stopper for the adaptation of the code.

4 List of requirements

This first step is crucial for the correct result. It is needed to define a clear and well defined list of requirements. This defines the main scope of the project, the different features to be implemented and prevents from building something that is not desired. This project needs to be able to fulfill the following requirements:

1. Application can be executed in CALCULA operating system (Linux)
2. Download measurements from the internet
   (a) Download measurements from a defined date range
   (b) Download measurements from a defined location
   (c) Download measurements from the Actris-Earlinet Data Portal [4]
      - Download ELPP products
      - Download Optical products
   (d) Download measurements from the Aeronet web [5]
      - Download Raw Almucantar products
      - Download Raw Polarized Almucantar products
      - Download Derived Inversion Products with Almucantar scenario
      - Download Aerosol Optical Depth (AOD) products
      - Download Total Optical Depth products based on AOD level
3. Filter Earlinet measurement files depending on the the data type: it is necessary that measurement has been done during day time, in order to have solar almucantar measurements. If not, these measurements should be discarded.
4. Execute pre-execution:
   (a) Read data of all downloaded files that are used as inputs for grasp algorithm
   (b) Preview data in order to choose a correct value for minimum and maximum heights
   (c) Obtain available configurations for the selected measure ID depending on available data
   (d) Choose a configuration
   (e) Generate a .sdat file that stores all data for selected measure ID and heights
   (f) Create .yml configuration file containing corresponding .sdat file name and chosen configuration
5. Execute the GRASP algorithm with corresponding configuration .yml file and .sdat file

6. Give the user the possibility to plot the different results after executing the algorithm
   - Detect all available measure IDs folders in output directory
   - Generate figure data files .mat from GRASP output data in selected measure ID output directory
   - Plot data from .mat files
7. Application can work with project/workspaces
   (a) User can create, open and import a project
   (b) Downloaded data from web services are stored in the project folder
   (c) Results of the GRASP algorithm are stored in the project folder
8. Application can have different configurations
   - Repository directories can be modified for both, earlinet and aeronet downloaded files
   - Output directory can be modified
   - GRASP installation directory can be modified: this path can be different for each user

5 Analysis and adaptation of existing tools

This step helps optimise both time and resources by preventing unnecessary work. Since this project builds upon a previous one, it is not required to develop new code for certain application requirements, but to adapt it to the new needs. Furthermore, the availability of APIs for downloading measurements from the ACTRIS–EARLINET Data Portal [4] and AERONET web application [5] provides a convenient and efficient solution that can significantly reduce development effort.

5.1 Data-portal services development

5.1.1 ACTRIS–EARLINET service

The ACTRIS-EARLINET Data Portal provides a REST API for downloading measurements [6]. This website shows all the different requests that are supported by the API, gives examples of how to use them, and the expected response.

Having all this information available is highly beneficial, since it allows one to locate the requests that are needed for the project requirements.

After testing and comparing the different options and the obtained responses, it becomes clear that the best option is to use the `/products/downloads` endpoint, which allows users to download the measurements in a compressed format, while filtering the data by type, date, and station name.

Even if other configuration parameters are available (measurementID, wavelength and opticaltype), it is not necessary to use them for the project requirements.

Once the behaviour is understood, a class is created to handle the requests and responses of the API.

A simplification of this class can be found in Appendix A.

It contains two methods of type `async Task<bool>` that allow the data to be downloaded from the API. When creating this class, it was taken into account that the `DownloadProductByDateRangeAsync` method can be used at different points in the program with various configurations. It has also been implemented so that adding new functionalities is straightforward using the same design pattern.

5.1.2 AERONET service

The AERONET website does not provide a REST API portlet, but it includes web service documentation that aids the implementation of a custom service.

The number of files to be downloaded from this website is larger than that of EARLINET, and its documentation can be found in three different sections of the website:

Table 1: AERONET Data Products and Extensions

Category	Product Description	Extension
Optical Depth [7]	Aerosol Optical Depth (AOD): (data description)	.lev15
	Spectral Deconvolution Algorithm (SDA): (data description)	.ONEILL_lev15
Aerosol Inversions [8]	Inversion products: (data description)	.all
Raw Products Optical Depth [9]	Raw Almucantar: (data description)	.alm
	Raw Polarized Almucantar: (data description)	.alp

When creating this class, consideration was given to the fact that the `DownloadProductByDateRangeAsy` method can be used at various points in the program with different configurations. It has also been implemented in such a way that adding new functionalities is straightforward using the same design pattern. For clarity and organisation, the `DataType` Enum has been created, since the different data types require different URL content to be downloaded. A simplification of this class can be found in Appendix B.

In this application, this class is only used to download the data files listed above, but other types could be downloaded by simply calling the `DownloadDataAsync` and `BuildUrl` methods.

5.2 Matlab controller development

As mentioned in the introduction, this project aims to automate the execution process of a set of scripts in Matlab.

The previously developed code has been adapted to the new hybrid system in order to be executed with C#. To achieve this, different actions have been taken.

5.2.1 Create script configuration files and output files

Since the project is hybrid, it is necessary to create a configuration file for the script that will be executed in C#. This file will contain the parameters needed to execute the script. In the previous approach, these files were not needed, since parameters introduced via the GUI were being stored in the Matlab workspace. The use of these files enables communication between the two different languages. Configuration files allow the different scripts to obtain the information and parameters input by the user, while the output files allow the application to retrieve the corresponding results and notify the user about it.

5.2.2 Automate Matlab repository for needed data files

Previously, it was necessary to manually download all the required data files; then, when the Matlab scripts were executed, a dialogue window would appear asking for the location of each file. For different data, a different number of files was required, having a maximum of five files to select. To automate this process, a script has been created that downloads the data files from the AERONET website and stores them in a local repository.

To automate the process of downloading and give the application the capability to work with different workspaces, the repository location has been modified and automated inside the application. The corresponding file is saved as a configuration parameter in both the application and the script.

5.2.3 Execute Matlab scripts from C#

To execute the Matlab scripts from C#, a method has been created that uses the `ProcessStartInfo` class, which is part of the standard library, to execute the Matlab script. Appendix C shows the `RunMatlabScript` method that takes a string parameter containing the path to the script to be executed. The method uses the `Process` class to execute the script and it redirects the standard output and standard error to the console.

5.2.4 Adapt old scripts to new file formats

Although there are some advantages to working with previous workspaces, it is necessary to adapt the old scripts to the new file formats. The main reason for this is the fact that the old scripts were created to work with a specific workspace, specific file names, and specific file structures. The new application is designed to work with different workspaces and to be updated to different file names and structures, since the downloaded files from the different websites do not maintain the format defined in the scripts.

It has also been necessary to adapt the different scripts to allow for different scenarios where some data may be missing or not available. In those cases, different messages have been added to inform the user during execution, and the corresponding output parameters have been defined for these cases.

Finally, unnecessary comments in the output window and unnecessary code have been deleted from the scripts, in order to provide a more professional experience for the final user.

6 Architecture

6.1 MVVM

Design pattern MVVM (Model-View-ViewModel) is a software architecture pattern designed with the goal of having a clear separation between the user interface (frontend) and the business logic (backend) [10]. There are different frameworks that use this standard, such as Angular, .NET WPF...

This application needs to be able to execute in CALCULA operating system (Linux), so it is necessary to use a framework that is compatible with this system. Originally, it was going to be developed using .NET WPF, but it was decided to use Avalonia instead, as it is a cross-platform framework that can be executed in different operating systems. The differences between both frameworks are very low, mainly being the name of the files (.axaml instead of .xaml) and the way to define the user interface.

This architecture is divided in three main components:

- Model: It contains the data and most of the app logic. It structures the information (classes and identities), how to retrieve it and how to manage it (services, data bases and controllers).
- View: It defines all the elements in the user interface and what the user will see and will interact with. The only responsibility of the code defined in this section is to define the visual structure of the app and to create input elements for the user to interact with.
- ViewModel: It is the bridge between the Model and the View. It is the one that controls the flow of data between the Model and the View.

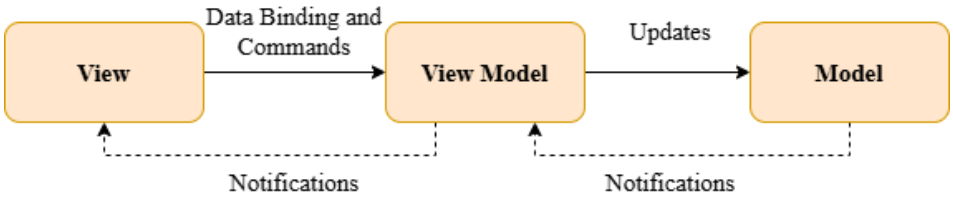


Figure 9: MVVM design pattern

The most important advantages of using this design pattern are:

- Changes only affect the view or the model, not both
- Separated testing for views and models
- User interface can be modified without affecting the model
- Teams with different developers can work on the same project simultaneously without affecting each other

6.1.1 Observable Object

In order to be able to update the user interface automatically when the data changes, Data Binding is used, which is a feature of the MVVM pattern. It is implemented using ObservableObject as a base class.

```
public class User : ObservableObject
{
    private string name;

    public string Name
    {
        get => name;
        set => SetProperty(ref name, value);
    }
}
```

In the example, it can be seen how a variable is updated, when it is detected that the new value is different from the old one, SetProperty is called, which implements INotifyPropertyChanged interface, which is the one in charge to notify the corresponding view.

In order that the view is updated, it is necessary to call the variable with the same name and to indicate that it is Bindable in the corresponding .axaml file, as it can be seen in the following example:

```
<TextBlock Text="{Binding␣Name,␣Mode=TwoWay,␣UpdateSourceTrigger=
    PropertyChanged}" />
```

In this case, the mode option indicates that the variable can be updated from the view to the model and vice versa, and the update source trigger option indicates that the update will be done when the property changes. Other options can be used, but these are the ones that are most common and that have been used in this project.

6.1.2 Relay Command

Relay command is used to implement commands in the MVVM pattern. It is implemented using ICommand as a base class. It can be used in different ways, but in this project it has been implemented using the following pattern for all View Models:

```
public class ViewModel : ObservableObject
{
  public ICommand Cmd => new RelayCommand(Execute, CanExecute);
  private void Execute(object _)
  {
      ExecutionActions();
  }

  private bool CanExecute(object _)
  {
      return true;
  }
}
```

In order to execute the command, it is necessary to link an interface element of the .axaml file to the Execute method

```
1 <Button Content="Execute" Command="{Binding␣Cmd}" />
```

6.2 Project architecture

Applying the MVVM design pattern and the SOLID principles[11], the project architecture can be summed up as follows:

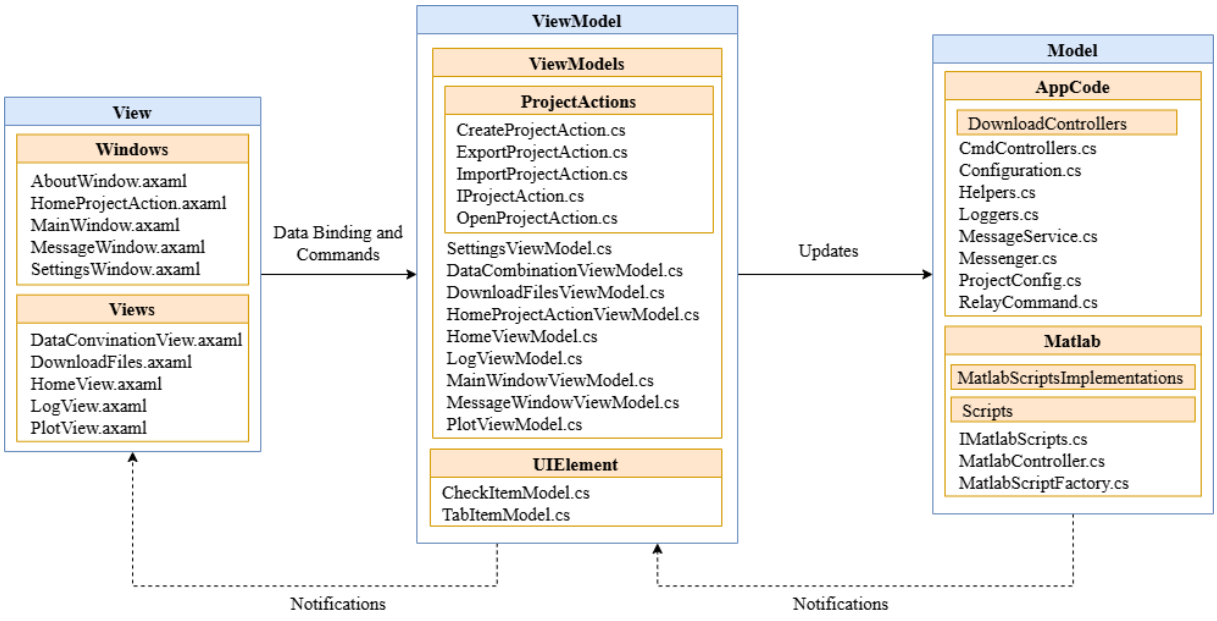


Figure 10: MVVM design pattern applied to the project architecture

In Figure 10 it can be seen how the Views are divided into different folders. The Windows folder contains all the files containing the definitions of the different windows and dialogs of the application, while the Views folder contains all the files containing the definitions of different UserControls. UserControls are used as Content in the MainWindow, in this case, as Contents of the different tab items.

The ViewModel section contains all the files containing the definitions of the different ViewModels of the application. Most of them are located in the ViewModels folder, but it can also be appreciated that there is a folder named UIElement, where the models for CheckItems and TabItems are located. With these classes, adding these items to Views and ViewModels is easier to maintain and update.

Finally, the Model component is composed of all the internal classes of the application. It has been divided into AppCode, where all the implementations related to app logic are located, and Matlab, where all the Matlab scripts are located as well as the necessary implementations for the app to execute them.

6.2.1 Matlab scripts

The Matlab scripts can be found in the project solution in the Scripts folder inside the Matlab one, as it can be seen in Figure 10. This folders, can be accessed also in the execution directory, with the same structure. This organisation can be seen in the following picture:

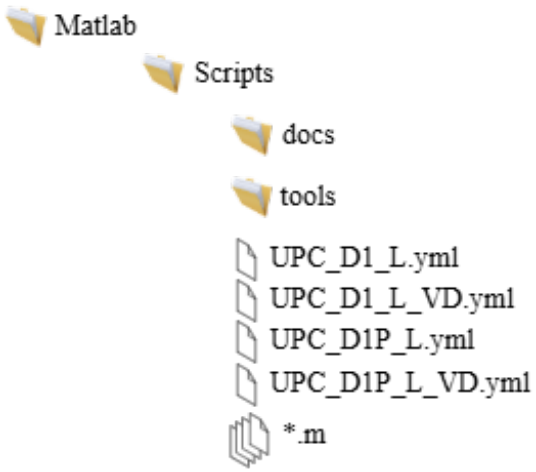


Figure 11: Matlab scripts organisation

Thanks to this, it is not necessary to compile the full project in order to modify the scripts or configuration files. If any modification of these files is needed, it can be directly accessed and modified. These scrips can be also compiled with Matlba without the need of executing the full application, but it is needed to configure different parameters, depending on the different scrips.

The most important files are:

- GetHeights.m: Calculates the minimum and maximum height of the indicated Measure ID
- Preview.m: Plots graph of the selected Measure ID of both Range Corrected Signal or Volume Depolarization
- SendData.m: Creates .sdat file that will be used for executing GRASP algorithm with the corresponding data of the selected Measure ID
- SaveFigures.m: Creates the different figures obtained from the output data after executing GRASP and saves them in the output folder in .mat files
- PlotFigure.m: Plots the selected figures from .mat files
- read_configuration.m: Reads the configuration file written by the main application during pre-execution actions
- .yml: Configuration files for GRASP. During execution, the corresponding file is copyied to the output folder and configured with the name parameters depending

on Measure ID and selected configuration

Other files can be found in this folder, containing functions that will be used in the different scripts mentioned in this list.

7 Graphic User Interface (GUI) design

In order to give to the user only the essential information, it was chosen to divide the different main actions that the application should perform into different views. In order to accomplish that, different UI implementations can be used. In this case, it was decided to implement a TabMenu, with three different views for each action: Download data files, Combine Data for GRASP algorithm execution and Plot GRASP results.

In this section, the design of the GUI will be described. A prototype of each view will be shown. In Figure 12, the corresponding view for the MainWindow can be seen.

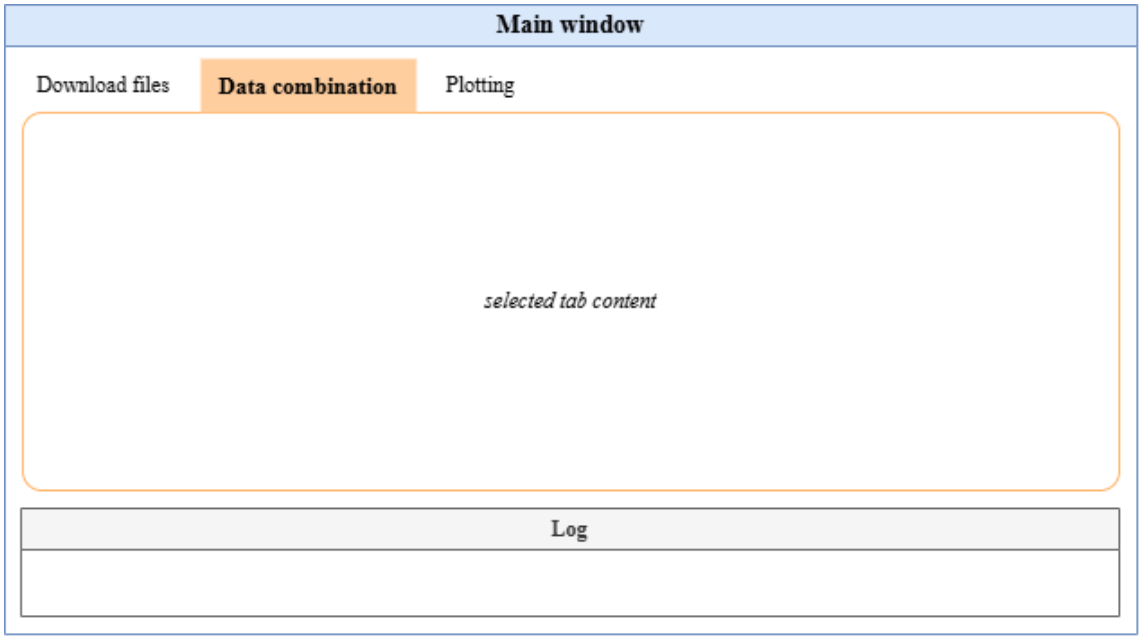


Figure 12: TabMenu design

With this window, the user will be able to access all the different functionalities of the application, which should be executed in the following order:

1. Download data files
2. Combine Data for GRASP algorithm execution
3. Plot GRASP results

Other necessary Windows were for Configuration, About and Message Windows. The content of these windows should be as simple as possible, since they are complementary windows and they do not contain a lot of information. For the Configurations window, it is only necessary to have one input element in order to introduce the value for each configuration parameter. Design can be modified taking into account the necessity of the current application.

When the application is first initialised, a Home View is needed, with different buttons,

where the user can choose which type of action they want to perform: Create, Open or Import project.

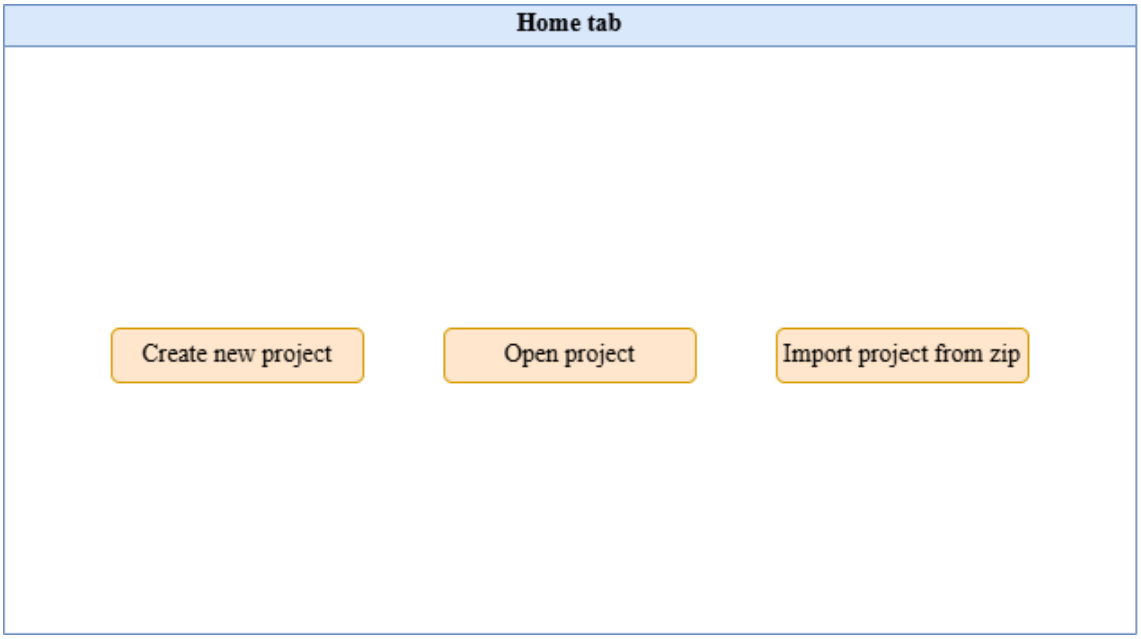


Figure 13: Home design

7.1 Download data files

In the requirements list, it was specified that the user should be able to download the data files within a selected date range and a selected station. After that, the user should be able to see a list of the downloaded files from each website.In order to implement this functionality, the first design uses two calendar selectors in order to choose "From date" and "To date". A text selector is used in order to select the station and a Download Button in order to start the execution.To finish, a list of the downloaded files is shown in a text block. All these features can be seen in the Figure 14.

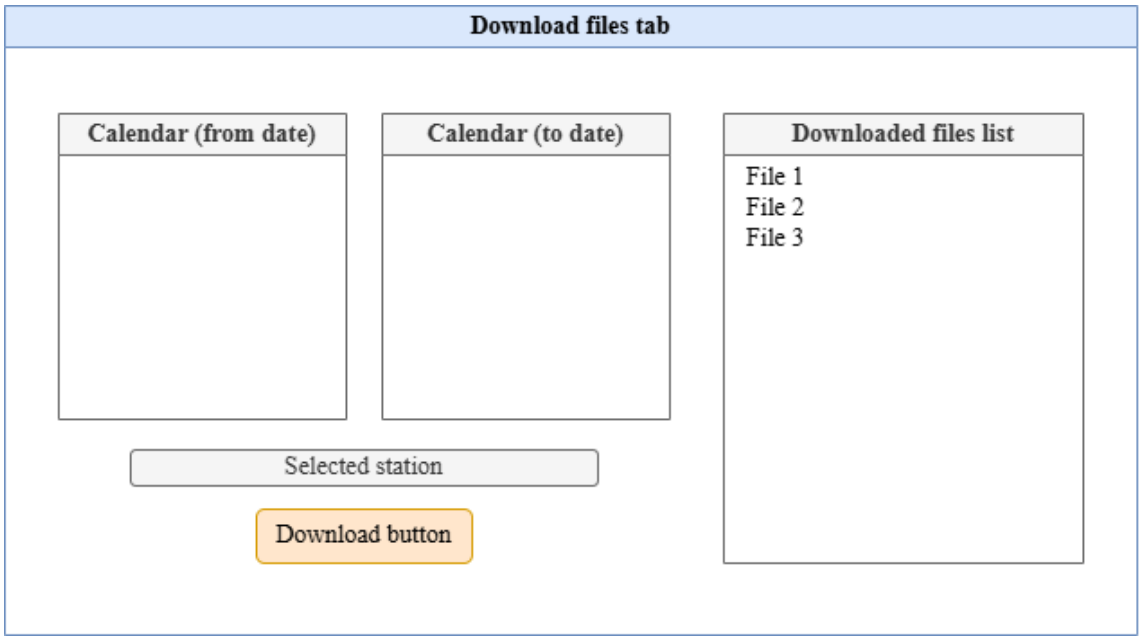


Figure 14: Download tab design

7.2 Data combination

For the data combination step, the user needs to select with which of the available measure IDs the GRASP algorithm is going to be executed. It is also necessary to have two options in order to select the minimum and maximum height for the data combination.

Once the data is previewed, it is necessary to notify the user of the different configurations that are available so that the user can choose one of them depending on the available data for the selected date and time.

Two buttons are used in order to start the different scripts execution: one for the data preview and the other one for generating the combination .sdat and .yml files and executing the GRASP algorithm execution.

All these features can be seen in Figure 15. In addition, a text section should be added in order to see the Output and Errors obtained from executing the different Matlab scripts used for pre-processing the data. Thanks to that element, the user will be able to see the process during the execution and the errors that may occur during it.

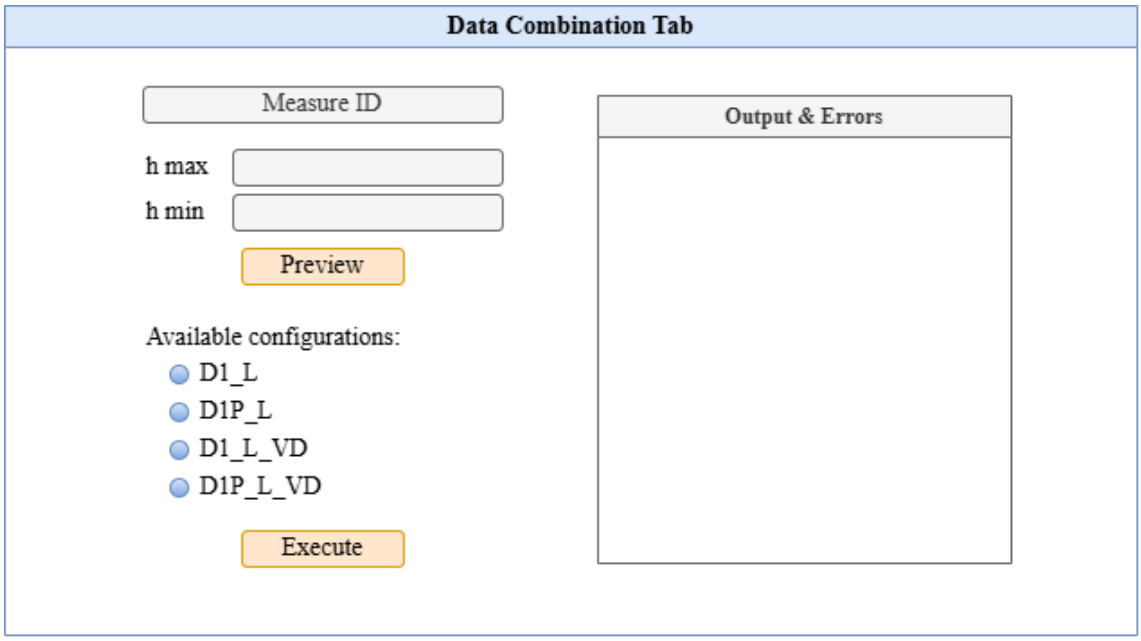


Figure 15: Data combination tab design

7.3 Plot GRASP results

For the last tab, the plotting tab, it is necessary to understand how the project folders are organised in order to know all the necessary elements that are going to be needed so that the user can introduce all the necessary information.As it can be appreciated in Figure 16, the project folder is organised in different subfolders, each one with a specific purpose. Data folder will contain all the downloaded data and Output data will contain the results of both, the data combination script and the GRASP algorithm execution.

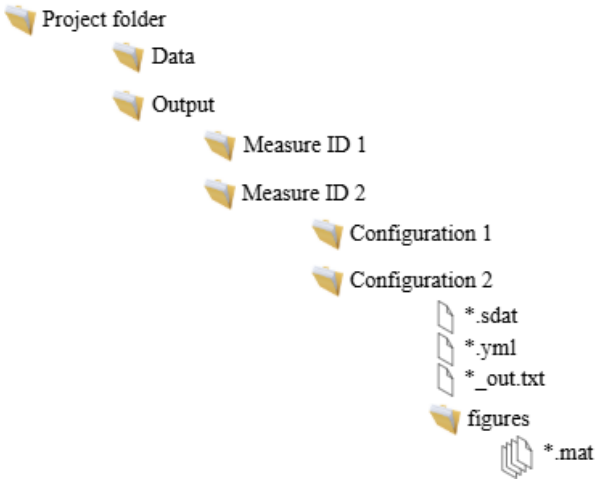


Figure 16: Project folders organisation

GRASP execution output is saved in a file named UPC_{config}_out.txt. This file will be the one used in order to create and save all the different files for each figure. Therefore, it is necessary first to specify the measure ID, then the measurement file and finally, the figure name that is desired to be seen.

These actions can be implemented with the different buttons and text inputs shown in Figure 17. Additionally, since this tab will also be executing Matlab scripts, it will be helpful for the user to see the progress and possible errors that can appear.

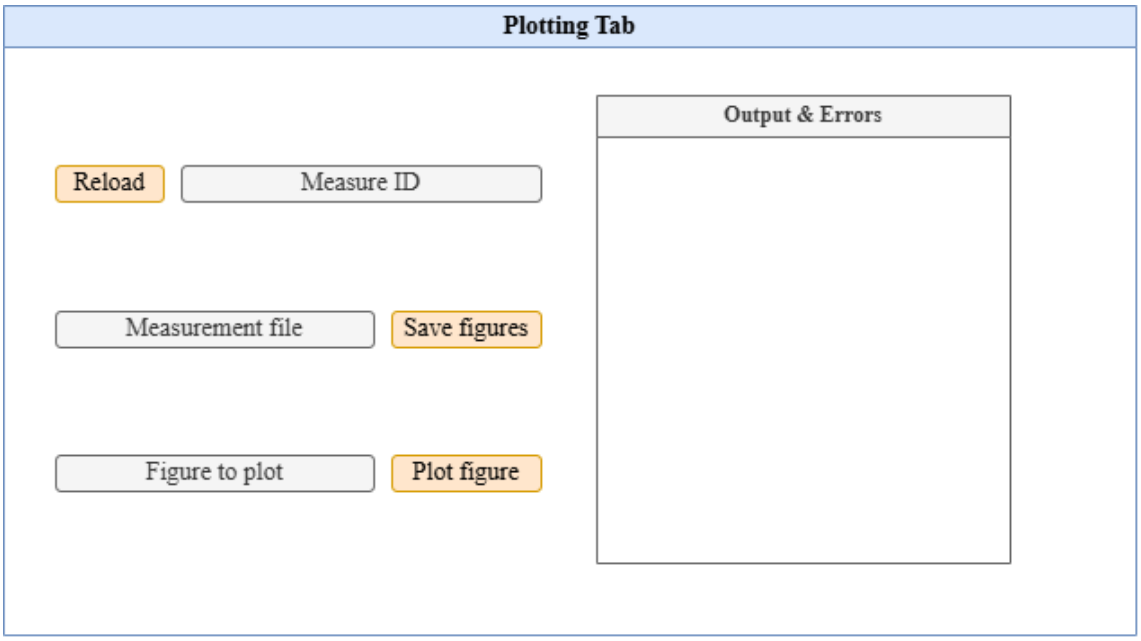


Figure 17: Plot tab design

8 Implementation

The code implemented for this project can be found in the GitHub repository[12]. Some implementations have been explained previously in other sections of this document like the Matlab Controller or the Aeronet Download Service, but this section aims to explain the purpose of the different classes and the different design patterns that have been used in order to follow the SOLID principles as much as possible.

8.1 Set up development environment

In order to create this project, it is necessary to meet several requirements to have a correct development environment. First of all, it is necessary to choose a correct IDE. There are different options, including JetBrains Rider and Visual Studio Code, but for this project, it has been decided to use Visual Studio Community [13].

The application will be run using .NET, which is a free and open-source application platform supported by Microsoft used for developing different type of applications using C# [14] and which is usually already installed in the system[15].

In order to use Avalonia, it is necessary to install the corresponding extension for Visual Studio. It can be done by accessing to Extensions¿ManageExtensions:

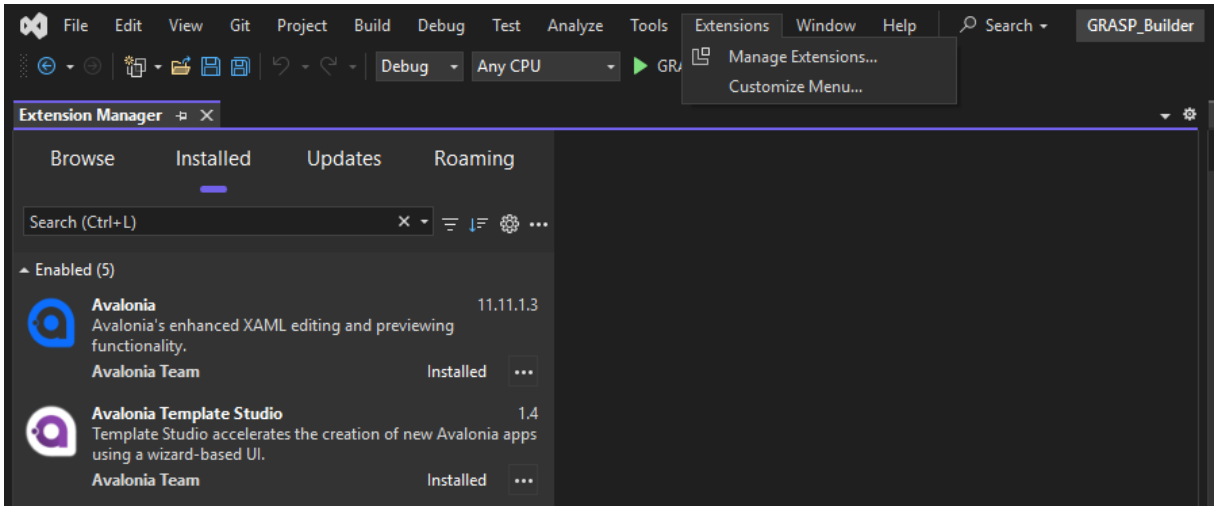


Figure 18: Extensions installed in Visual Studio

Since the app will be executing Matlab scripts, it is also necessary to have Matlab installed. In this case it has been used Matlab R2024a[16].

In order to run the GRASP algorithm it is necessary to access to CALCULA, where it is already installed and running in Ubuntu that will only be used for testing.

8.2 Views and ViewModels

In order to not overload the content of the different files for the User Interface implementation (Views and Windows) and management (View Models) it has been created a different View class for each Tab and for each Window, that can be resumed in the following list:

- About Window: Window that shows general information about the app and containg button "Help" which opens User Manual
- Data Combination View: Tab where the user can select Measure ID to combine data, showing available configurations for selected date and time and executing GRASP.
- Data Download View: Tab where the user can select dates and station and download corresponding data from Earlinet and Aeronet websites.
- Home View: Shows the following action options: to create, open or import a project.
- Home Project Action Window: Window that appears when the user clicks on a project action option in Home View and is in charge of performing it.
- Main Window: Window that contains all the tabs and shows them when necessary. It can show the Home View when no project is loaded or a combination of the other tabs with an additional Log and a Menu in order to manage the app Settings or the current project.
- Message Window: Window that shows a message to the user that can be an Error, a Warning or just information.
- Log View: Tab that shows the Log content of the app.
- Plot View: Tab where the user can select and create the desired figures after executing GRASP algorithm.
- Settings View: Window where the user is able to modify the app configuration parameters.

All these classes for both Views and ViewModels are implemented using the MVVM pattern using ObservableObject and RelayCommand classes from the .NET community toolkit. In addition, all ViewModels classes have been structured by regions in order to homogeneously group the different properties, commands and methods.

In order to establish communication between different classes to notify modifications, it has been used the Messenger class from the .NET community toolkit. It consists on defining a Send Action, that can include some parameter or variable and a Receive Action, that will be executed when the Send Action is called, executing the linked method.

In the following example it can be seen an example of how the DataCombinationViewModel is using the Messenger class to receive modifications in the buttons enabled state:

```
1 public DataCombinationViewModel()
2 {
3     Messenger.Default.Register<bool>("UpdateButtonsEnabled",
          UpdateButtonsEnabled);
```

```
}

private void UpdateButtonsEnabled(bool status)
{
    AreButtonsEnabled = status;
    if (!status)
    {
        isEnabled_D1_L = false;
        isEnabled_D1P_L = false;
        isEnabled_D1_L_VD = false;
        isEnabled_D1P_L_VD = false;
    }
}
```

In the corresponding controller, in order to send the modification in order to enable the buttons, for example, it is necessary to execute the following line:

```
Messenger.Default.Send<bool>("UpdateButtonsEnabled", true);
```

Using Messenger allows to notify modifications in the App between classes that exist simultaneously, in this case between the different Tabs and between ViewModels and Controllers. In the case of creating new Windows like the Settings one, it is not necessary, since the needed information can be passed as parameters to the Constructor when it is initialized.

8.3 Controllers

With the objective of dividing the different actions and separating the different responsabilities, the following list of Controllers have been developed.

- Download Controllers: Different controllers have been developed for each different web site. They both implement the IDownloadController Interface. Each controller is responsible for the different actions that need to be executed when downloading the respective data by managing their respective web services.
- CmdController: This controller is responsible for the different actions needed to execute commands, in this case, necessary to execute the GRASP algorithm via the command line. Since this application have been designed to work with CALCULA, command line for executing GRASP only is executed with Ubuntu syntax.
- AppConfig: This class manages the application configuration, obtaining and saving the different configuration parameters from the configuration file (config.json), located in the execution directory.
- Helpers: Different helper classes have been developed to perform different actions. FormatHelpers are used for changing the format of variables, for example: Converting a dictionary to a string variable. FileHelpers are used for managing the files and directories renaming, copying, etc.
- Logger: This class is responsible for managing the logging actions.

- MessagesController: This controller is responsible for managing the different messages that can be shown to the user: Errors, warnings and information messages.
- ProjectConfig: This class manages the project configuration, obtaining and saving the different configuration parameters from the project file (project.grasp), located in the project directory.
- MatlabScriptController: This controller is responsible for managing the different actions needed to execute any MatlabScript.

8.4 Polymorphism through interfaces

With the objective of applying the Open/Closed Principle (OCP) and the Interface Segregation Principle (ISP), different interfaces have been implemented in order to be used in different points of the project. Using polymorphism, enables the possibility of using objects of a derived class as objects of a base class[17].

Polymorphism can be implemented with a parent or base class, which defines the common properties and implements virtual methods. Derived classes can use the already implemented methods of the parent class, or override them.

Another possible implementation of polymorphism is through the use of abstract classes or interfaces, which is the one used in this project. Interfaces are contracts that define a set of methods that a class must implement. This enables the possibility to work with different classes, where each class implements the interface in a different way but all classes and actions are used uniformly by calling the interface methods.

8.4.1 Factory pattern

Factory pattern is a creational design pattern that provides an interface for creating objects of a parent class, but deciding which subclass to instantiate[18]. This allows to resume all the different conditions in one single method, instead of using multiple if-else conditions during the development of the code.

An example of this pattern can be seen in the following example, showing the implementation of the DownloadControllerFactory Create method:

```
1  public static IDownloadController Create(DownloadType type, string
       _repositoryDirectory, string _workingDirectory)
2  {
3      switch (type)
4      {
5          case DownloadType.Earlinet:
6              return new EarlinetDownloadController(_repositoryDirectory,
                   _workingDirectory);
7          case DownloadType.Aeronet:
8              return new AeronetDownloadController(_repositoryDirectory,
                   _workingDirectory);
9          default:
10             throw new NotSupportedException($"Download␣type␣{type.
                   ToString()}␣is␣not␣supported");
11     }
```

```
}
```

8.4.2 Project Actions

The first usage of this coding strategy can be found in IProjectAction interface. This interface is used when initializing a project when the application is opened, where the user can choose between the different actions that can be executed: Create, Open or Import from .zip file. Export action has also been implemented, but it is also accessible through the File Menu, once a project is loaded.

```
public interface IProjectAction
{
    public string Title { get; set; }
    public string ProjectName { get; set; }
    public string DirectoryPath { get; set; }
    public bool IsProjectNameVisible { get; set; }
    public bool IsDirectoryPathVisible { get; set; }
    public Task<bool> Execute();
    public Task Browse();
}
```

This interface defines five different properties, that are used for initializing the View and ViewModel before showing the new window, and two methods, that correspond to the different actions that can be executed in this window: Browse a directory and Execute.

With this implementation, IProjectAction object is used in HomeProjectActionViewModel. Only in the constructor the type of action needs to be specified. In the rest of the class, all the Bindings and Commands are executed using this object equally for all the types. A simple example is the implementation of Execute command.

```
public ICommand BrowseCmd => new RelayCommand(async _ => await
    BrowseExecute(), CanBrowse);

private bool CanBrowse(object _)
{
    return true;
}

private async Task BrowseExecute()
{
    _projectAction.DirectoryPath = DirectoryPath;
    _projectAction.ProjectName = ProjectName;

    await _projectAction.Browse();

    DirectoryPath = _projectAction.DirectoryPath;
    ProjectName = _projectAction.ProjectName;
}
```

In this Command, it can be appreciated the simplicity of the code and the lack of if-else conditions and knowledge of the type of action that is being executed.

8.4.3 Download Controllers

Another class that can take advantage of the use of polymorphism is the DownloadController, which needs a simpler interface.

```
public interface IDownloadController
{
    public string RepositoryDirectory { get; set; }
    public string WorkingDirectory { get; set; }
    public Task Download(DateTime FromDate, DateTime ToDate, string
        station);
}
```

With this implementation, the only method that needs to be called is Download, but in each implementation, this method execute different actions.

For the moment, only two classes are implemented using this interface, but in the future, if it was desired to add any other data source, it would be possible to implement a new class that implements this interface and used it in the ViewModel as it is done now with the IDownloadController object.

```
IDownloadController downloadController = DownloadControllerFactory.
    Create(DownloadType.Earlinet, _earlinetRepositoryDirectory,
    _workingDirectory);
await downloadController.Download(FromDate, ToDate,"BRC");
```

8.4.4 Matlab Scripts Implementations

The last interface that has been implemented is IMatlabScript. Running Matlab scripts using C# is not a complicated task, since the use of the library ProcessStartInfo speeds up all the process, needed only the name of the script to be executed. But for this project it was required to execute more actions.

```
public interface IMatlabScript
{
    public string Name { get; }
    public Dictionary<string, object> vars { get; set; }
    public void PreExecutionActions();
    public void PostExecutionActions(bool resultOK = true);
}
```

It was needed not only to run a script but to have communication between the Matlab scripts and the application. In order to solve this problem, it was decided to create different files:

- Configuration files: Parameters are saved from C# application and read in Matlab before running the corresponding script.
- Output files: Results are saved from Matlab and read in C# application after running the corresponding script, in order to update the UI or execute further actions.

This is very useful, since it allows to have a clear separation between the different tasks and to make the code more maintainable. If it is desired to add a new script, it is only necessary to create a new class using this same interface and call the method for running a Matlab script as in the following example:

```
MatlabController.RunMatlabScript(ScriptType.Preview, dict,"_DC");
```

This is possible because this interface is only used for running Matlab Scripts. The MatlabController class is responsible for executing all the sequence for running a script, which consists on:

1. Execute pre-execution actions
2. Run the script (start matlab process)
3. Receive output from Matlab
4. Receive results from Matlab
5. Execute post-execution actions

Each Script implementation will be the responsible for defining the different actions to be executed in each step, taking into account if the script finished with error or was executed successfully.

8.5 Interactions with other applications

The main goal of this application is to create a user-friendly interface for executing all the sequence of actions needed to download data from different web services, combine it and create plots from the results. In order to do that, the application needs to interact with different external applications and web services:

1. Web services: in order to download all the necessary data files from the different sources, Aeronet and Earlinet sites
2. Matlab: in order to execute the different scripts needed for data combination and plotting
3. GRASP algorithm: in order to execute the algorithm and obtain the results

With the use of the different interfaces and classes explained in this section, it is possible to obtain the desired behaviour, which can be summarized in the following diagram:

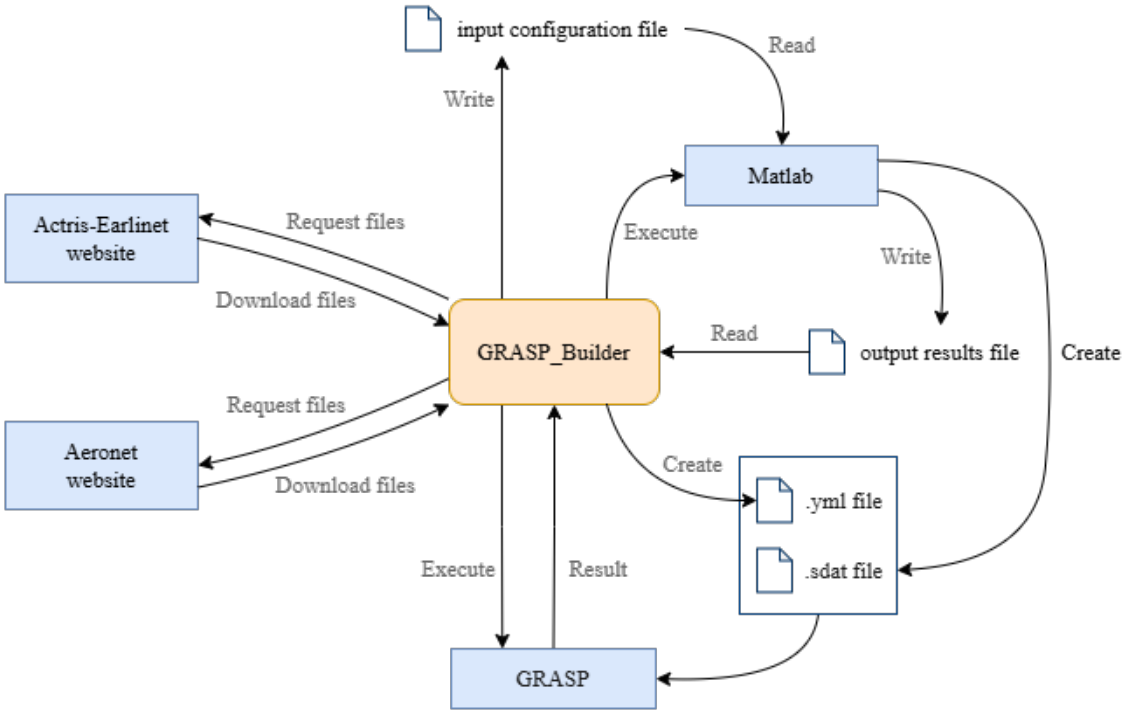


Figure 19: Application interactions diagram

In Figure 19 it can be appreciated all the different actions that are taking place during the execution of the application and the communication between the application and the external applications and web services. All these interactions are not noticed from the user point of view, giving the impression of an easier user experience.

8.6 Code organization

Since this projects has been developed for the CommonSense department, all classes have been organized in different regions in order that the code is easy to understand and maintain for a new developer.

```
namespace GRASP_Builder.ViewModels
{
    3 references
    public class DataCombinationViewModel : ViewModelBase
    {
        Constructor

        Messenger

        Members

        Binding

        Commands
    }
}
```

Figure 20: Code organization example

In Figure 20 all the different regions of any View Model class can be appreciated.

- Constructor: Only contains the class Constructor method.
- Messenger: Contains all the related methods to the different Messenger Register cases that have been defined in the corresponding class.
- Members: Contains all the variables and properties of the class. These variables can be accessed by all the class methods.
- Binding: Contains all the binding properties implemented for the corresponding View.
- Commands: Contains all the commands implemented for the corresponding View.

In addition, different summaries and comments have been added to the code in order to make it easier to understand. The source code can be downloaded from the following repository: `https://github.com/mireialopez0910/TFM/tree/main` . In this repository it is included the GRASP_Builder C# project and both LaTeX projects, the User Manual document and the Master Thesis document, and the User Manual document in PDF format.

Git can be installed through the official website: `https://git-scm.com/download/win`. Once it is installed, the repository can be cloned in the desired folder by executing the following command in the desired terminal:

```
1    git clone https://github.com/mireialopez0910/TFM.git
```

9 Testing

During the development of the application, different tests have been performed to ensure that the application works as expected:

1. First contact:
   In order to know how the initial implementation was working and optimise the process it was needed to know how the different steps were being executed and the expected results
   - Download files
   - Execute Matlab scripts with old data
   - Execute GRASP with old data
2. Test downloaded data: After automating the download process, it was needed to test the different matlab scripts with the new data. During these tests, it was detected that some data was being downloaded with a new file format. This was a problem, since the old scripts were not compatible with this new format and they had to be modified.
3. Test GRASP with generated data: After modifying the scripts, it was needed to test the obtained data with the GRASP algorithm. It was detected that the data was not being processed correctly since the configuration file did not match the correct values. This was an issue with file versions that was solved by updating the configuration file to the correct version.
4. Test full UI execution: After defining the application as configurable and workspace independent, it was needed to test the full execution of the application. During this round of tests, it was checked that the project folders were working as expected, improving the project management process.
5. Test full UI execution in CALCULA: Since CALCULA works with LINUX different aspects of the application had to be tested. Some changes were needed for the folders and files management, since some routing was only working in Windows. During these tests the full execution of the application was performed successfully, since it was the only test where the GRASP algorithm could be executed. The full testing results table can be found in Appendix D.
6. Test different dates with problematic scenarios: After checking the full execution of the applications, it was needed to test the application with negative tests. Negative test consist on testing the software in known and problematic scenarios, in order to check if the application is able to handle them correctly. The main cases checked were:
   - Missing data for wavelength 355 or 532 nm: GRASP can not be executed with any configuration
   - Missing data for wavelength 1064 nm: GRASP can be executed with all configurations

- Missing VD data: GRASP can not be executed with D1_L_VD and D1P_L_VD configurations
- Missing ALP data: GRASP can not be executed with D1_P_L and D1_P_L_VD configurations
- Missing AOD data: GRASP can not be executed with any configuration
- Missing ALL data: GRASP can not be executed with any configuration
- Missing ALM data: GRASP can not be executed with any configuration

Unit testing has also been performed to ensure that the application is working as expected. This type of testing consists in checking the separated components of the application, in this case the Model classes, to ensure that they are working as expected[19].

Implementing this type of testing in a project allows to detect and fix some issues that were not detected during the integration tests and reduces the time consumption of some testing process.

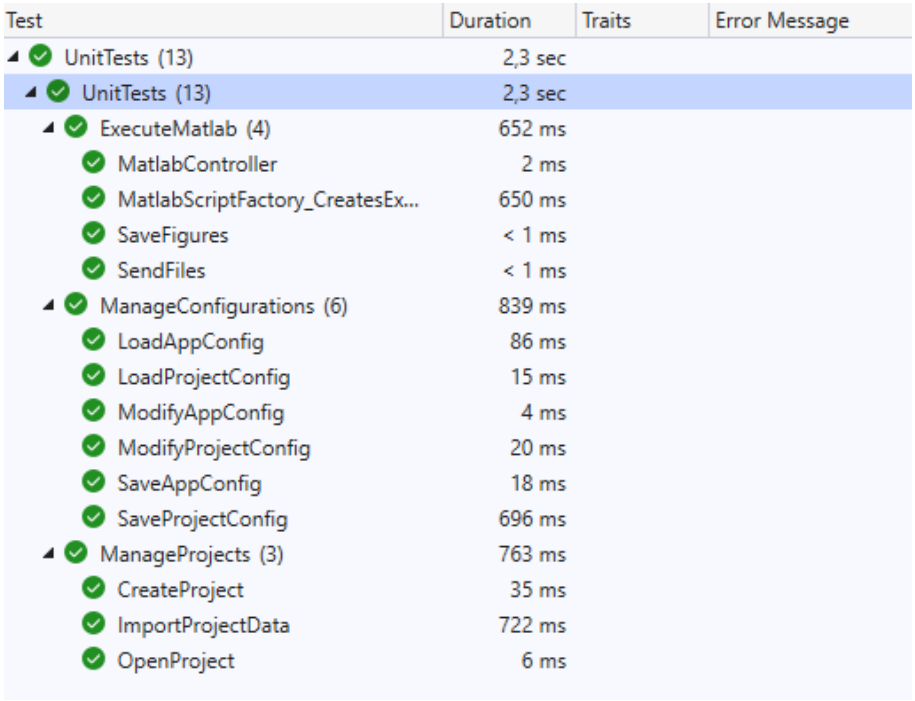


Figure 21: Implemented Unit Tests

In Figure 21 it can be seen the implemented unit tests for this project, where different classes have been tested: application configuration, project configuration and Matlab script management. The main advantages of this type of testing are that they can be performed efficiently, in this case they have been executed within seconds, and can be performed during development. This helps the developer to check that the new code is still working as expected.

10 Final product

This section describes the released version of the application and the final execution sequence, although a more detailed explanation can be found in the User Manual, in Appendix E. The application executables can be found in `https://drive.google.com/drive/folders/1-ah8Bmy-WjuZU2IpYdQzZkzaAbUyykzX?usp=sharing` inside GRASP_Builder_v200_Li

10.1 Final User Interface

During the development of the application, the user interface has been modified to be more user-friendly and to add more features that were not contemplated during the design phase.

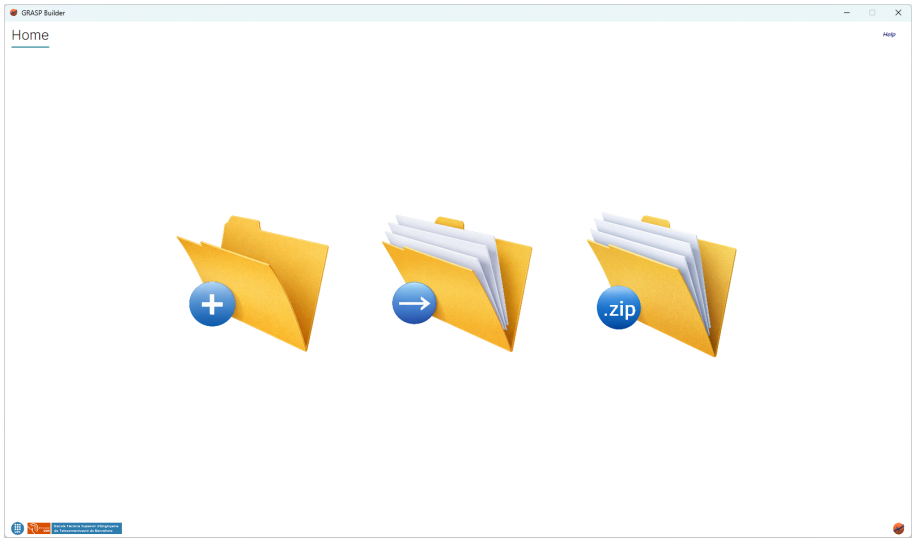


Figure 22: Home view

The Home view, which is shown when the application is initialised, allows the user to select the different options available in the application: Create, Open or Import project. This can be seen in Figure 22. It has also been added the help button, which opens the User Manual in .pdf format.

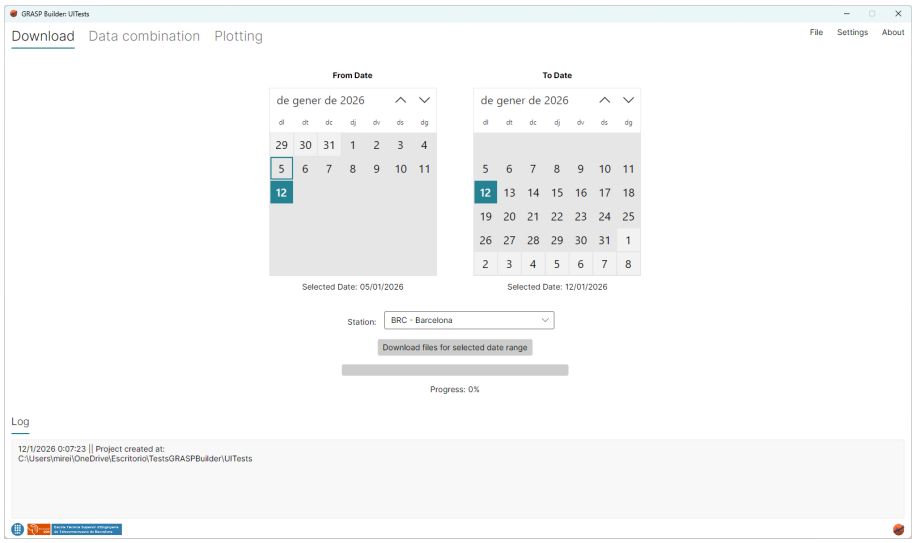


Figure 23: Download view

The Download View maintains mostly the initial design. Since the process of downloading files can depend on the user's internet connection, it has been added a progress bar to show the progress of the download.

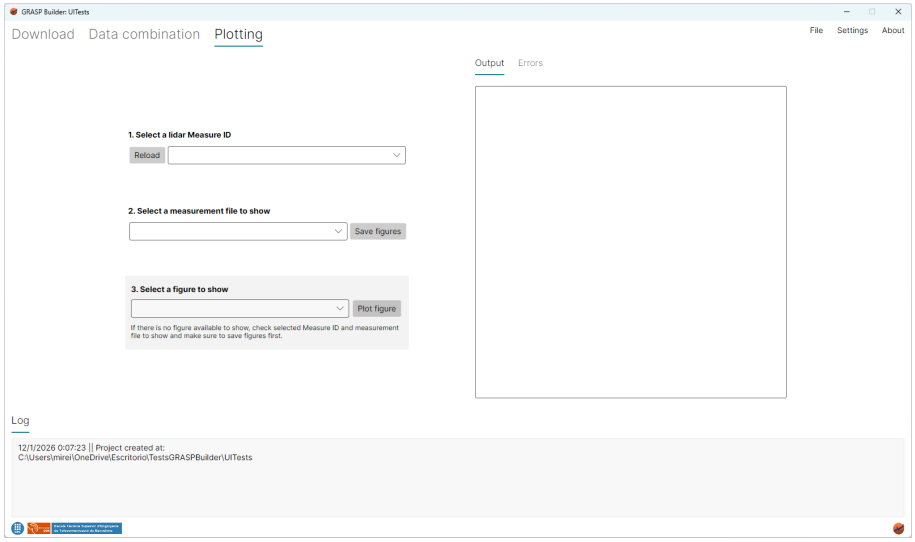


Figure 24: Data Combination view

Data Combination has not been modified from the initial design, since it allows the user to select the different files to be combined and the different parameters to be used for the combination. This can be seen in Figure 32.

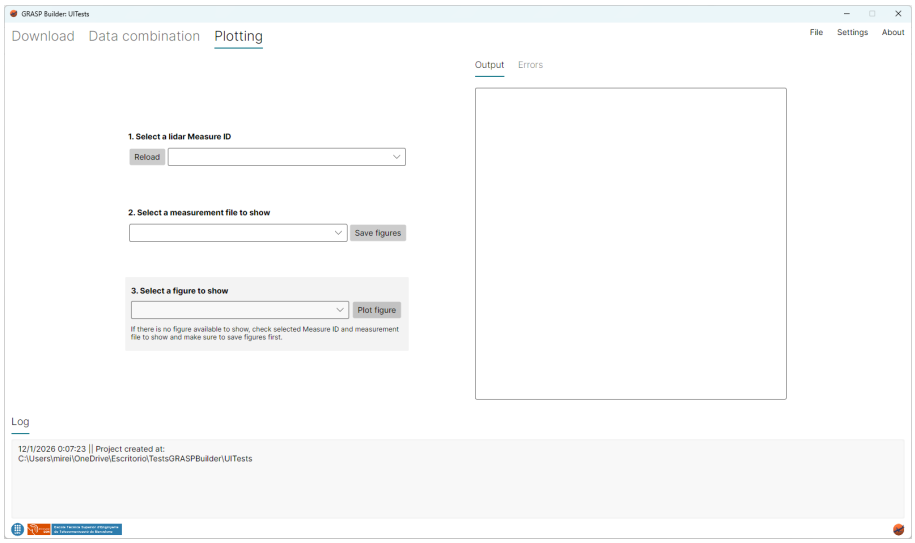


Figure 25: Plotting view

Plotting view has not been modified from the initial design either, since it allows the user to select the different files to be plotted and the different parameters to be used for the plotting, shown in Figure 33.

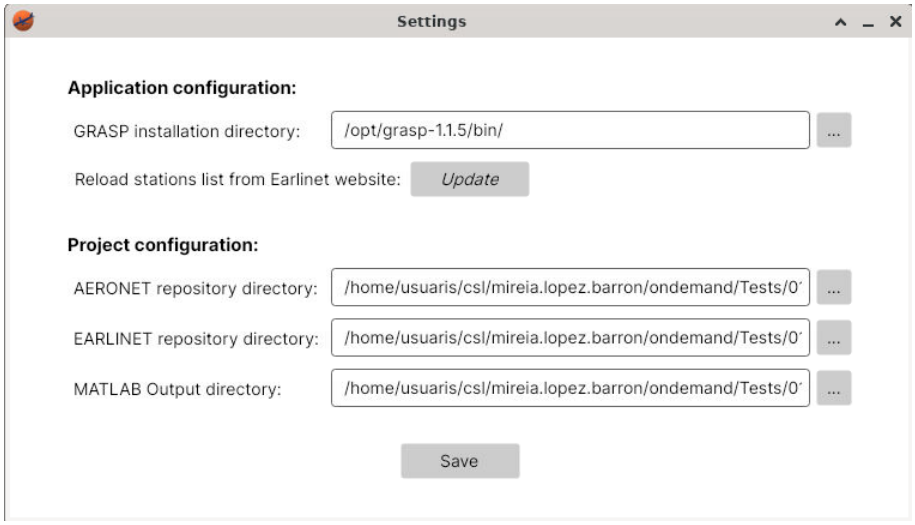


Figure 26: Settings window

Settings window has been created in order to allow the user to configure both the application and project settings, shown in Figure 26. This type of configuration has been clearly separated, in order to let the user know which modifications will affect only the project and which ones will affect all executions of the application, independently of the project loaded.

10.2 Work-flux

The final work-flux for working with GRASP algorithm can be summarised in the following steps, all included in the developed application and executed in CALCULA environment:

1. Create, open or import a project: Load a project in order to save downloaded data and ouput data and figures in a defined directory.

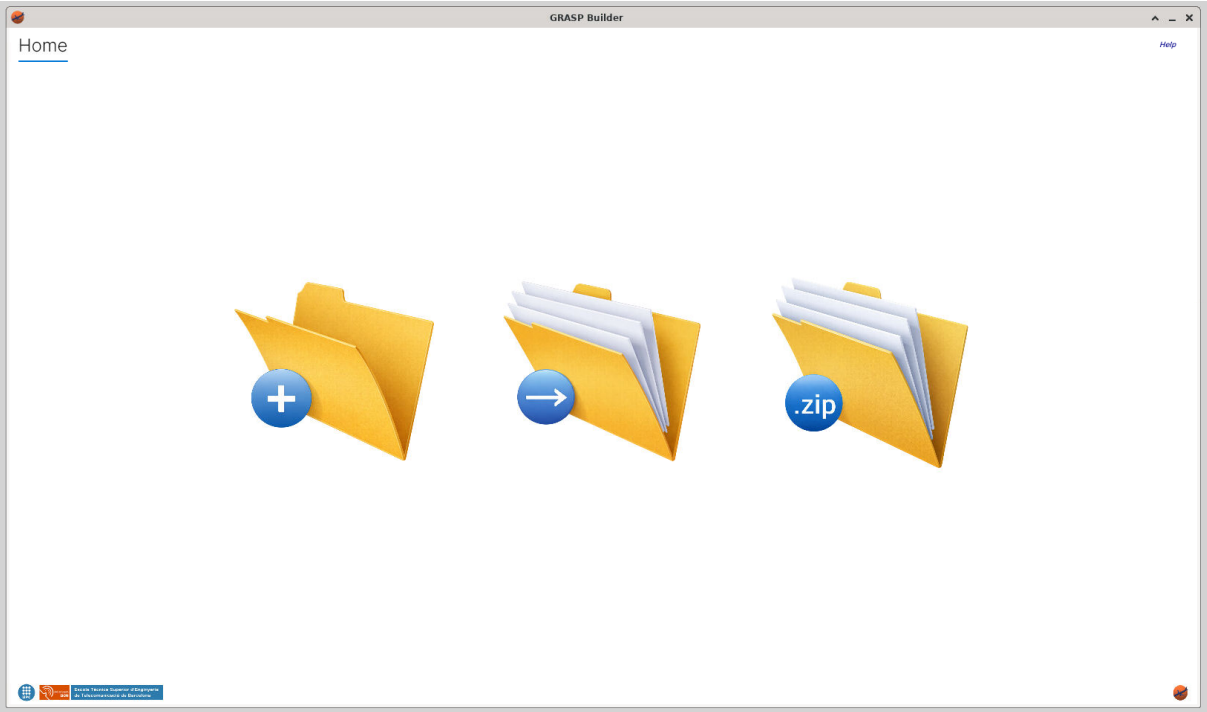


Figure 27: Home screen

2. Download files: Select date range and station and download the corresponding files. Available files will be shown in same View.

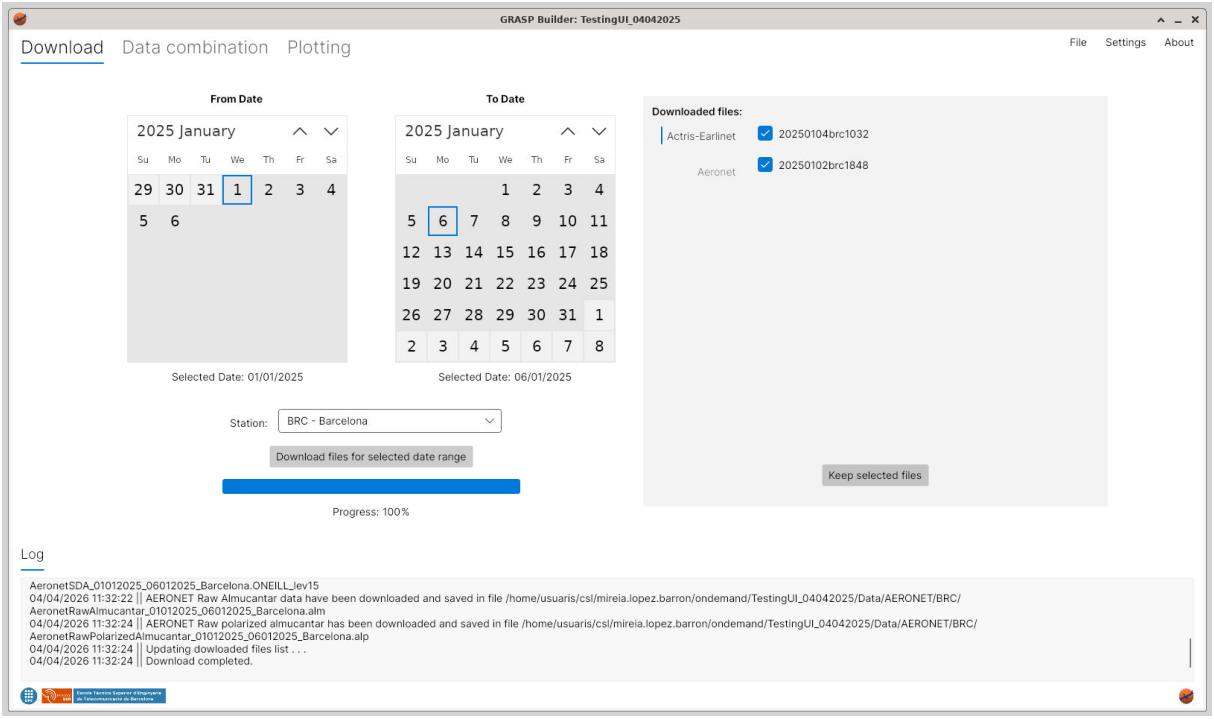


Figure 28: Files downloaded

3. Combine data: Select the desired MeasureID, choosing the data and time range to be used. Heights can be configured and available configurations will be automatically shown.

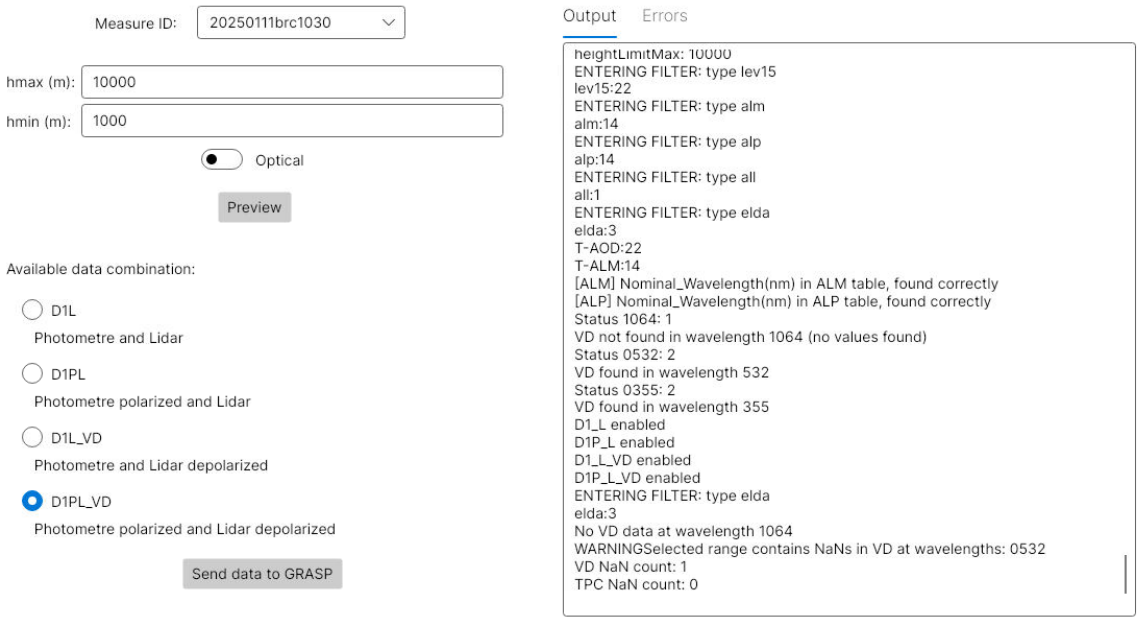


Figure 29: Data combination with enabled options

4.

5. Execute GRASP: Send data will create the input files automatically for GRASP algorithm and will execute it.

6. Plot results: Available results will be automatically detected. If possible, list of figures will be created and saved in Output directory. The user will be able to choose which figure to show.

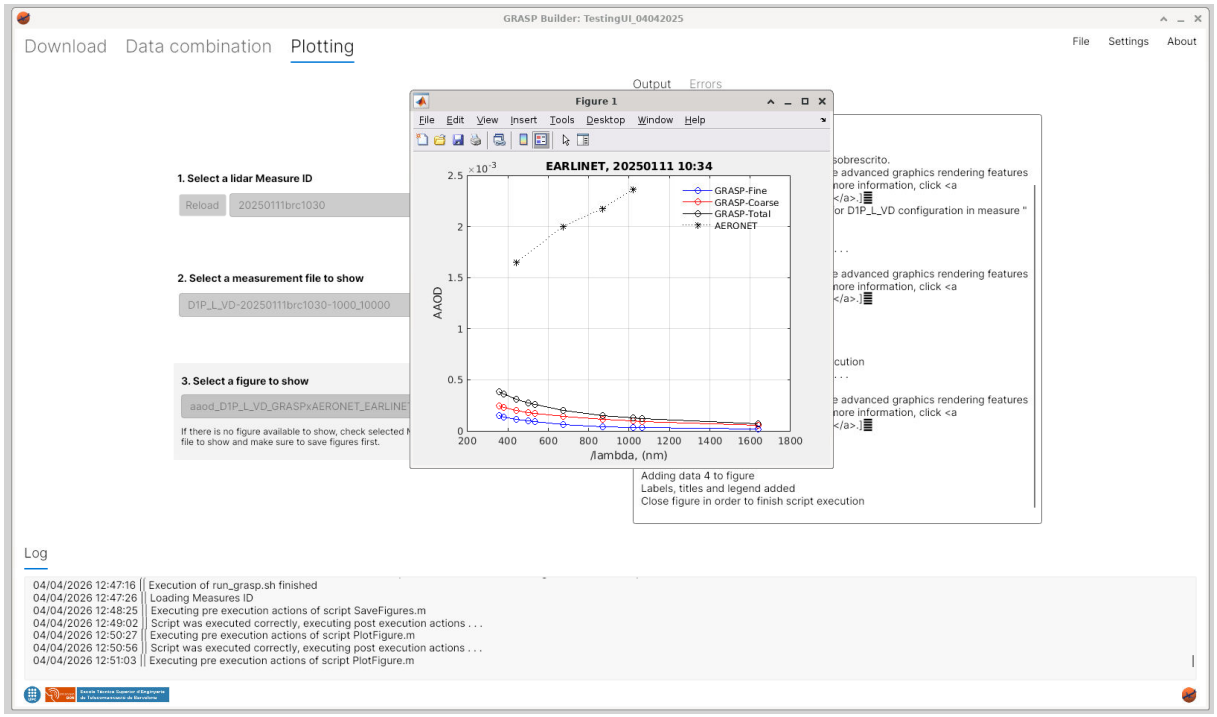


Figure 30: Plotted figure example

10.3 Low resolution User Interface

Using Avalonia Nuget package provokes the inability to resize the application window, since it does not maintain the correct format for the moment. In some tests, it has been detected that all actions cannot be executed in a low resolution system. Then, it has been created the option of a low resolution UI, which can be downloaded from the same link with name GRASP_Builder_v200_Linux_Small.zip in `https://drive.google.com/drive/folders/1-ah8Bmy-WjuZU2IpYdQzZkzaAbUyykzX?usp=sharing`. This option allows the user to execute the application in a low resolution system, having a slightly modified UI, which can be seen in the following figures: **??**

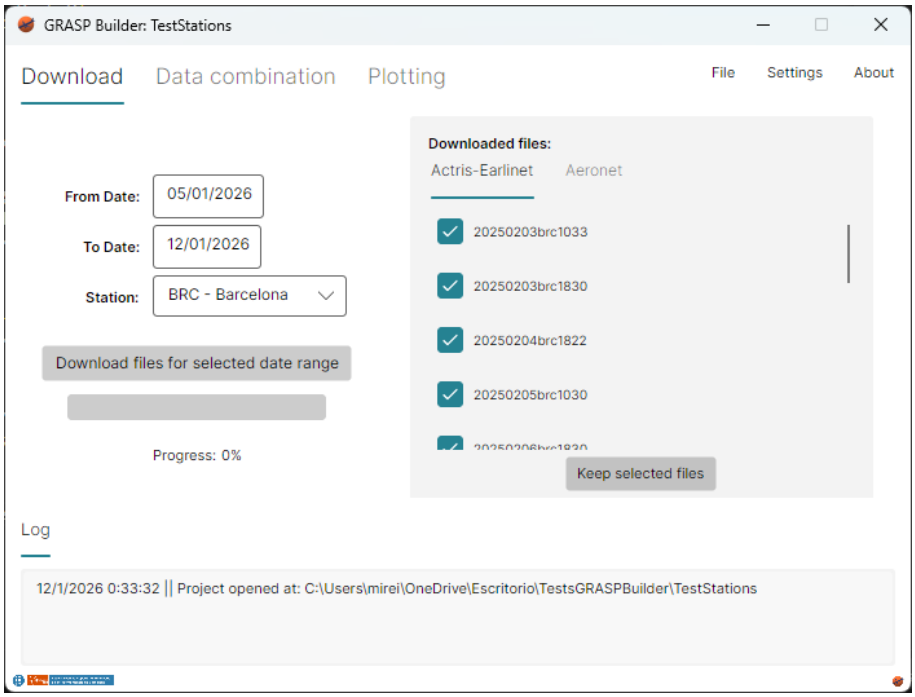


Figure 31: Download view for low resolution UI

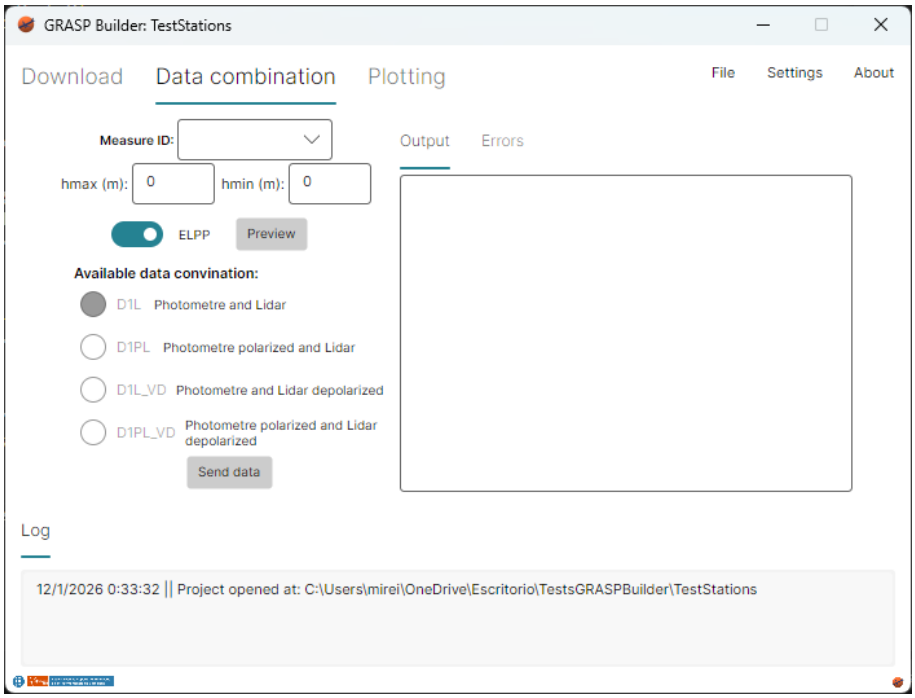


Figure 32: Data Combination view for low resolution UI

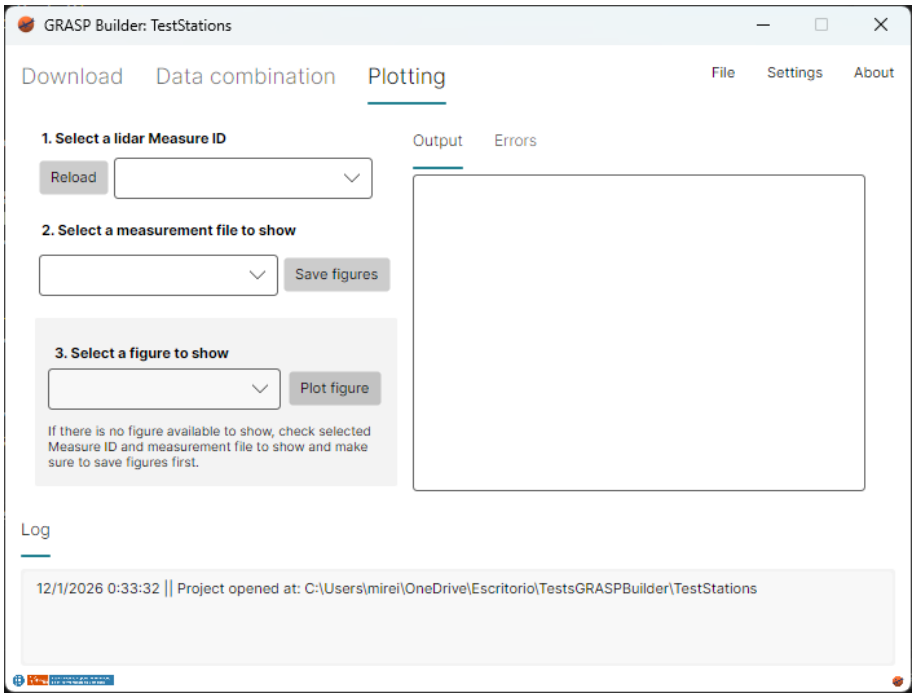


Figure 33: Plotting view for low resolution UI

This option allows the user to execute the application in a low resolution system, since this User Interface is smaller than the lowest screen resolution option in Windows operative system.

11 Conclusions

The result of this project is a reliable and easy-to-use application that automatically performs all the steps of the GRASP algorithm almost in real time. After reviewing the whole process from the initial ideas to the final testing, here are the main conclusions:

Main Objectives Achieved: The main goal was to connect all the steps together in one automatic process, and this has been fully achieved. The application can now easily download data from AERONET and ACTRIS-EARLINET. Then, it runs the Matlab scripts to combine the data, executes the GRASP algorithm, and finally shows the results in a graph. All this automation saves time for the user and makes data analysis much faster.

Working on Different Systems and Handling Errors: The decision to use C# and the Avalonia UI package was very important. Thanks to this, the application works perfectly in the CALCULA system, which uses Linux. Also, the program is very strong at handling errors. Because different testing methods were used, such as unit testing and negative testing, the application can deal with problems, like missing data or changed file formats, without crashing.

Better and Clearer Code: At the beginning of the project, it was complicated to understand the old code because it was not organized and the file formats were outdated. However, this problem was solved by creating a more logical and structured program. By organizing the code following design patterns and good practices, it was also possible to create unit tests. This means that if someone else wants to work on this project in the future, it will be much easier to understand and improve.

Better User Experience: The new application is designed specifically for the people who will use it. It has a clear interface that separates the main application settings from the specific project options, making everything simpler. Furthermore, there is an option to use a smaller interface designed for computer screens with a low resolution. This ensures that anyone can use the program, regardless of the computer they have.

Importance of Methodology and Requirements: Finally, this project has shown how important it is to follow a clear methodology and have a well-defined list of requirements from the start. Because the initial requirements were not very clear, there were many unexpected problems and changes during the development. By eventually defining a strong plan and clear goals, it became much easier to solve problems, organize the code, and finish the application successfully. This proves that spending time on good planning and setting requirements is essential for any software project.

References

[1] GRASP Open. Grasp. Web Application available at https://www.grasp-open.com/, 2025.

[2] Miriam Martínez Canelo. ¿qué es la programación orientada a objetos? Web Application available at https://profile.es/blog/que-es-la-programacion-orientada-a-objetos/, 2025.

[3] MathWorks. Matlab. Web Application available at https://es.mathworks.com/products/matlab.html, 2025.

[4] C.N.R. IMAA. Actris-earlinet data portal. Web Application available at https://data.earlinet.org/earlinet/desktop.zul, 2024.

[5] NASA. Aerosol robotic network (aeronet). Web Application available at https://data.earlinet.org/earlinet/desktop.zul, 2025.

[6] C.N.R. IMAA. Actris-earlinet rest api. Web Application available at https://data.earlinet.org/api/swagger-ui/, 2025.

[7] NASA. Aerosol optical depth. Web Application available at https://aeronet.gsfc.nasa.gov/print_web_data_help_v3_new.html, 2025.

[8] NASA. Aerosol inversions. Web Application available at https://aeronet.gsfc.nasa.gov/print_web_data_help_v3_inv_new.html, 2025.

[9] NASA. Raw products aerosol optical depth. Web Application available at https://aeronet.gsfc.nasa.gov/print_web_data_help_v3_raw_sky_new.html, 2025.

[10] Microsoft. Mvvm. Web Application available at https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm, 2025.

[11] GeeksforGeeks. Solid principles. Web Application available at https://www.geeksforgeeks.org/system-design/solid-principle-in-programming-understand-with-real-life-examples/, 2025.

[12] GitHub. Github. Web Application available at https://github.com/mireialopez0910/TFM/tree/main, 2025.

[13] Microsoft. Download visual studio community. Web Application available at https://visualstudio.microsoft.com/es/vs/community/, 2025.

[14] Microsoft. What is .net? Web Application available at https://dotnet.microsoft.com/en-us/learn/dotnet/what-is-dotnet, 2025.

[15] Microsoft. Download .net. Web Application available at https://dotnet.microsoft.com/en-us/download/dotnet, 2025.

[16] MathWorks. Download matlab. Web Application available at https://es.mathworks.com/help/install/ug/install-products-with-internet-connection.html, 2025.

[17] Microsoft Learn. Polymorphism. Web Application available at https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/polymorphism, 2025.

[18] Microsoft Learn. Factory pattern. Web Application available at https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/, 2025.

[19] AWS. Unit testing. Web Application available at https://aws.amazon.com/es/what-is/unit-testing/, 2025.

Appendices

A Earlinet Service Class

```
public class EarlinetService{
    string baseUrl = "https://api.actris-ares.eu/api/services/restapi/";
    public virtual System.Threading.Tasks.Task<bool>
        DownloadProductByDateRangeAsync(string type,string FromDate,
        string ToDate,string outputFolder){
         return DownloadProductByDateRangeAsync(type,FromDate,ToDate,
             outputFolder,System.Threading.CancellationToken.None);
    }

    public async Task<bool> DownloadProductByDateRangeAsync(string type,
        string FromDate,string ToDate,string outputFilePath,System.
        Threading.CancellationToken cancellationToken = default){
         try{
             string url = $"{baseUrl}products/downloads?kind={type}&
                fromDate={FromDate}&toDate={ToDate}&stations=brc";
             using (HttpClient client = new HttpClient()){
                 HttpResponseMessage response = await client.GetAsync(url
                     );
                 response.EnsureSuccessStatusCode();
                 using (var fs = new FileStream(outputFilePath,FileMode.
                     Create,FileAccess.Write,FileShare.None)){
                      await response.Content.CopyToAsync(fs);
                 }
                 System.IO.FileInfo f = new FileInfo(outputFilePath);
                 if (f.Length <= 1048)
                      return false;
                 return true;
             }
         }
         catch (Exception ex){
             return false;
         }
    }
}
```

B Aeronet Service Class

```
public class AeronetService{
    private string baseUrl = "https://aeronet.gsfc.nasa.gov/cgi-bin/";
    public async Task DownloadDataAsync(string destinationFile,string
        url){
         try{
             using (HttpClient client = new HttpClient()){
                 var response = await client.GetAsync(url);
                 response.EnsureSuccessStatusCode();

```

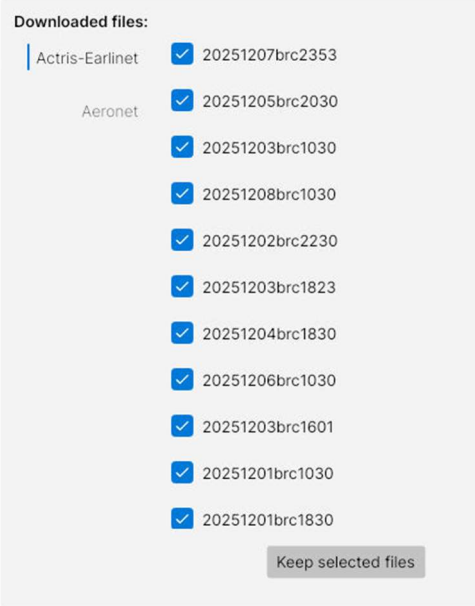
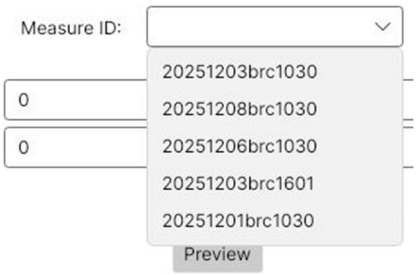
```
                var content = await response.Content.
                    ReadAsByteArrayAsync();
                await File.WriteAllBytesAsync(destinationFile,content);
            }
        }
        catch (Exception ex){
            Console.WriteLine($"Error downloading data: {ex.Message}");
        }
    }

    public string BuildUrl(DataType _dataType,DateTime startDate,
        DateTime endDate,string productType = "",string site = "",string
        product = "",string AVG = "",bool isEnabled = false){
        switch (_dataType){
            case DataType.AerosolInversions:
                if (isEnabled)
                    return BuildUrlAerosolInversions(startDate,endDate,
                        productType,site,product,AVG);
                else
                    return BuildUrlAerosolInversions(startDate,endDate);

            case DataType.OpticalDepth:
                if (isEnabled)
                    return BuildUrlOpticalDepth(startDate,endDate,
                        productType,site,AVG);
                else
                    return BuildUrlOpticalDepth(startDate,endDate,
                        productType);

            case DataType.RawProductsOpticalDepth:
                if (isEnabled)
                    return BuildUrlRawProductsOpticalDepth(startDate,
                        endDate,productType,site,AVG);
                else
                    return BuildUrlRawProductsOpticalDepth(startDate,
                        endDate,productType);
        }
        return "";
    }

    {...} //BuildUrl methods can be found in GitHub repository
}
```

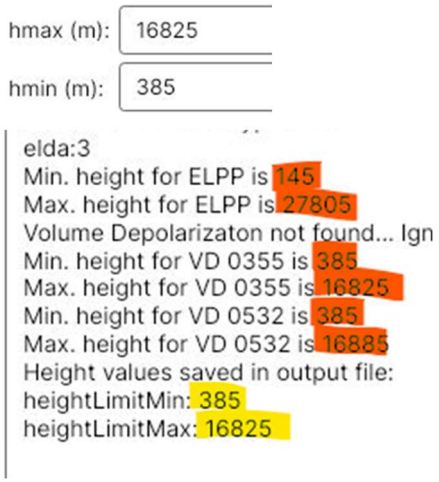
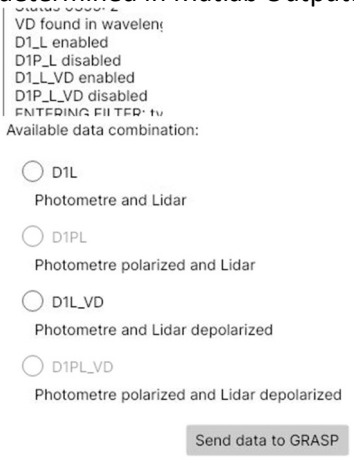
C RunMatlabScript method using ProcessStartInfo class

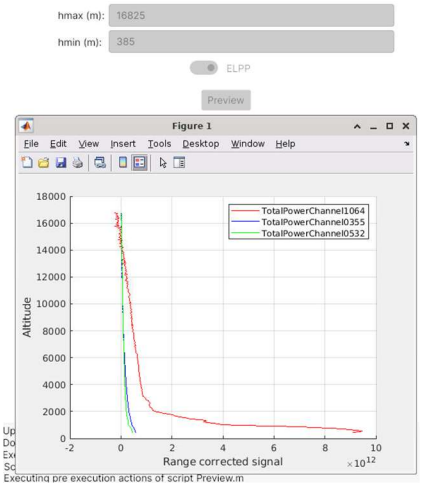
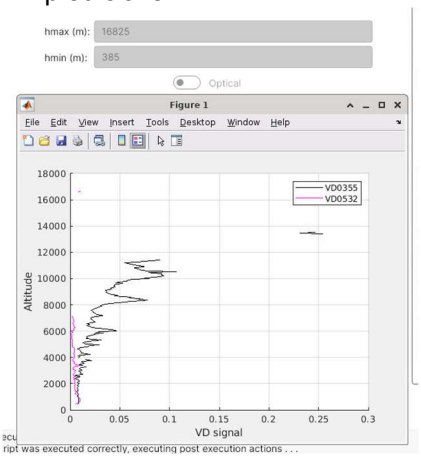
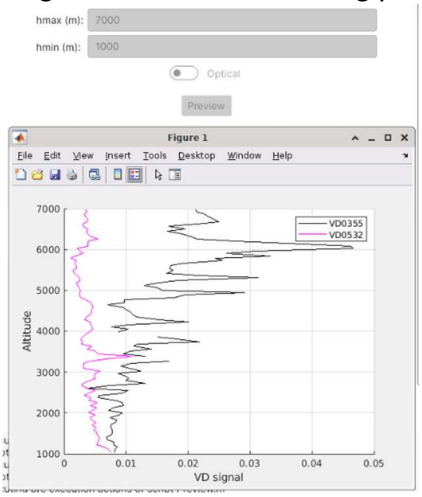
```
public static void RunMatlabScript (string scriptPath){
    try{
        ProcessStartInfo startInfo = new ProcessStartInfo{
            FileName = "matlab",
            Arguments = $"-batch \"run('{scriptPath}')\"",
            RedirectStandardOutput = true,
```

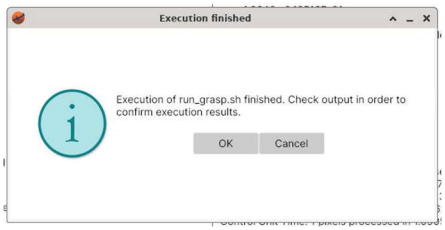
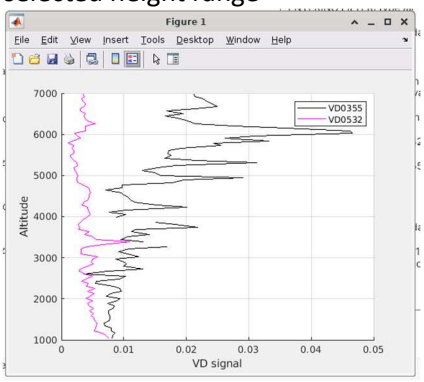
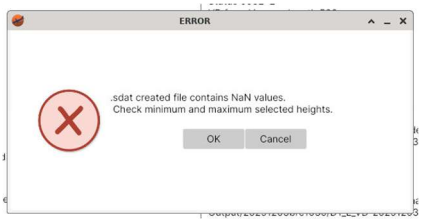
```
            RedirectStandardError = true,
            UseShellExecute = false,
            CreateNoWindow = true
        };
        using var process = new Process { StartInfo = startInfo };
        process.OutputDataReceived += (s,e) =>{
            if (e.Data != null)
                Messenger.Default.Send<string>("WriteMatlabOutput",e.
                    Data);
        };
        process.ErrorDataReceived += (s,e) =>{
            if (e.Data != null)
                Messenger.Default.Send<string>("WriteMatlabErrors",e.
                    Data);
        };
        process.Start();
        process.BeginOutputReadLine();
        process.BeginErrorReadLine();
        process.WaitForExit();
        if (process.ExitCode == 0){ //OK
            Logger.Log($"Script was executed correctly, executing post
                execution actions . . .");
            script.PostExecutionActions();
        }
        else{
            Logger.Log($"Error occured during execution, executing post
                execution actions . . .");
            script.PostExecutionActions(false);
        }
    }
    catch (Exception ex){
        Logger.Log($"Error occured during execution, executing post
            execution actions . . .");
        script.PostExecutionActions(false);
    }
}
```

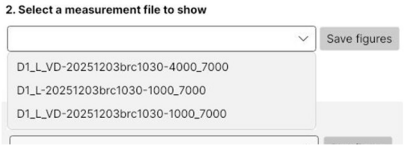
D Test Results

#	Test	Expected result	OK/KO
1	***In Download view:*** Download files from 01/12/2025 to 08/12/2025	When files are downloaded message is shown in log view during Progress is lower than 100% If timeout is reacherd for downloading files message is shown in log view Once progress is 100% all downloaded files are listed	OK
2	***In Download view:*** If timeout is reached during download, download a smaller date range	All files are downloaded correctly	
3	***In Download view:*** Uncheck files in list and click button keep selected files	Files that have been unchecked disappear from list and are deleted from project's data folder	
4	***In Download view:*** Keep files from diferent times, for exemple:  Move to Data combinaton view	In available measure IDs list only files that have day time measurements appear In the current exemple:  	OK
5	***In Data combination view:*** Select Measure ID	GetHeights.m script executes automatically in order to determine mínimum and màximum heights	OK
6	***In Data combination view:*** During GetHeights.m execution	All buttons are disabled Messages are shown in log: 1. Executing pre execution acitons 2. Executing post execution actions Matlab script Output is being written simultaneously	OK

#	Test	Expected result	OK/KO
7	***In Data combination view:*** GetHeights.m execution finishes	Buttons are enabled Hmax and Hmin value are automatically defined with script calculated height 	OK
8	***In Data combination view:*** Click Preview button	Preview.m script executes	OK
9	***In Data combination view:*** During Preview.m exeuction	All buttons are disabled Messages is shown in log: Executing pre execution acitons Matlab script Output is being written simultaneously	OK
10	***In Data combination view:*** During Preview.m exeuction: before closing figure	All buttons keep disabled Executing post execution actions message is not shown	OK
11	***In Data combination view:*** During Preview.m exeuction:after closing figure	All buttons are enabled Post execution actions message appears in log Configurations get enabled as it is determined in Matlab Output: 	OK

#	Test	Expected result	OK/KO
12	***In Data combination view:*** Execute Preview.m with ELPP option	ELPP plot is shown: 	OK
13	***In Data combination view:*** Execute Preview.m with ELPP option	VD plot is shown 	OK
14	***In Data combination view:*** Modify lengths and execute Preview.m	Plot axis minimum and maximum lengths are modified accordingly: 	OK
15	***In Data combination view:*** Select a configuration ans click “Send data to GRASP” button	SendFiles.m executes	OK

#	Test	Expected result	OK/KO
16	***In Data combination view:*** During SendFiles.m exeuction	All buttons are disabled Messages are shown in log: 1. Executing pre execution acitons 2. Executing post execution actions Matlab script Output is being written simultaneously	OK
17	***In Data combination view:*** SendFiles.m finishes executing	Messages are shown in log: 1. SaveOuputNameInConfigFile message 2. Starting GRASP execution 3. Results will be saved...	OK
18	***In Data combination view:*** GRASP execution finishes	Information window is shown: notifies the user execution finished  All files are correctly saved in Ouput folder for choosen configuration	OK
19	***In Data combination view:*** Send data to GRASP with NaN values on selected height range 	Error is shown 	OK
20	***In Plotting view:*** Click Reload button	Measure ID that have GRASP results in folder are shown as options 	OK

#	Test	Expected result	OK/KO
21	***In Plotting view:*** Select lidar Measure ID Check available measurement files	Only measurement files for this Measure ID are shown All measurement files for this Measurement ID are shown 	OK
22	***In Plotting view:*** Select a Measurement file of a successfull GRASP execution and click button “Save Figures”	Script SaveFigures.m is executed	OK
23	***In Plotting view:*** During SaveFigures.m is executing	All buttons are disabled Figures appear in screen while they are being created and are saved in corresponding folder	OK
24	***In Plotting view:*** After SaveFigures.m has executed	All figures appear listed as option in point 3 “Select figure to show”	OK
25	***In Plotting view:*** Select a figure and click “Plot figure” button	PlotFigure.m is executed Figure is shown Buttons are dissabled until Figure is closed	OK
26	***In Plotting view:*** Select a Measurement file of a unsuccessfull GRASP execution and click button “Save Figures”	In both Output and Log is shown error information: Output:  Log: 	OK

E User Manual

GRASP Builder User Manual

Master Thesis
submitted to the Faculty of the
Escola Tècnica d'Enginyeria de Telecomunicació de Barcelona
Universitat Politècnica de Catalunya
by

Mireia López Barrón

In partial fulfillment
of the requirements for the master in
TELECOMMUNICATIONS **ENGINEERING**

Advisor: name of the advisor
Barcelona, Date XXXXX

Contents

1 Set up

GRASP Builder application can be executed in both operative systems, Windows and Linux. The only prerequisite in order to work with this application is to have MATLAB installed.

The only functionality not enabled in Windows is the execution of GRASP algorithm, since this application is desigend to work in CALCULA enviroment, but all the other funcionalities can be used, if the user prefers.

1.1 Windows

Since this application is not being installed with any executable or saved in registry, it is only necessary to unzip file *GRASP_Builder_v200_Windows.zip* in the desired folder.

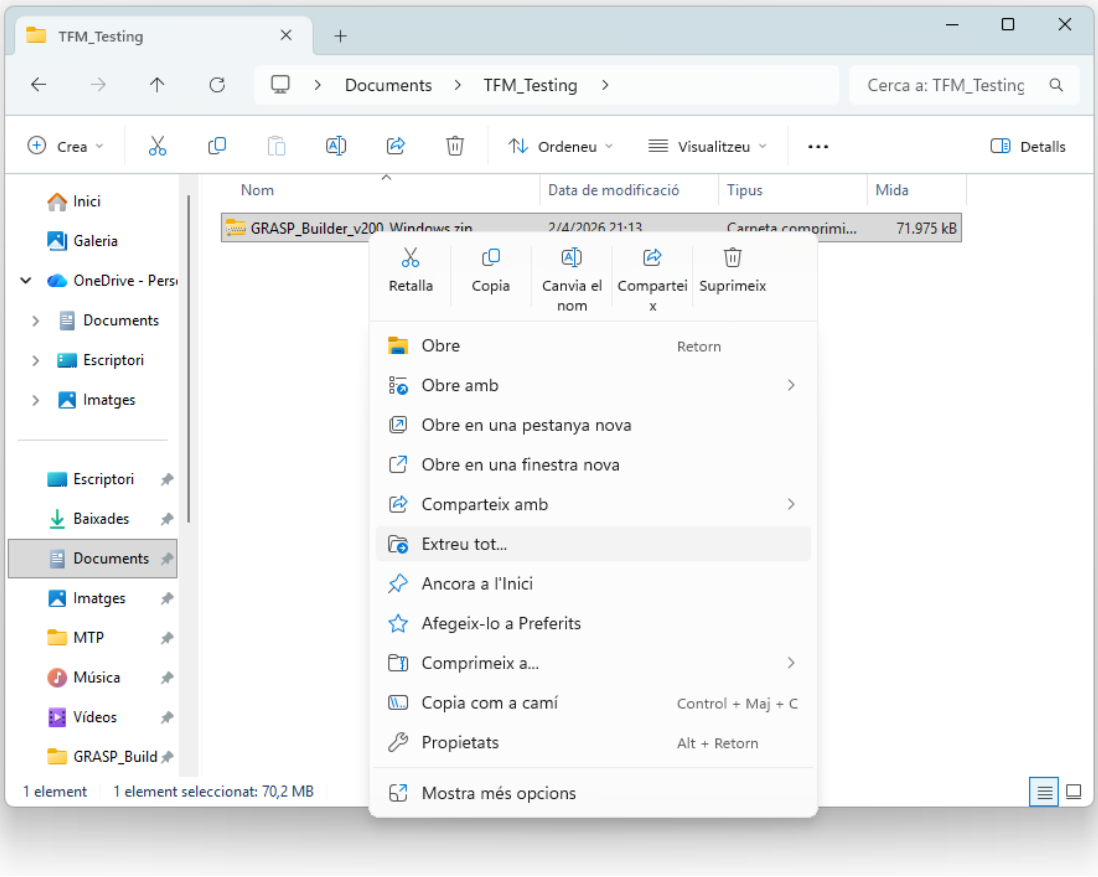


Figure 1: Unzip process in Windows

In the unziped folder it will be located and executable called GRASP_Builder.exe and it can be executed.

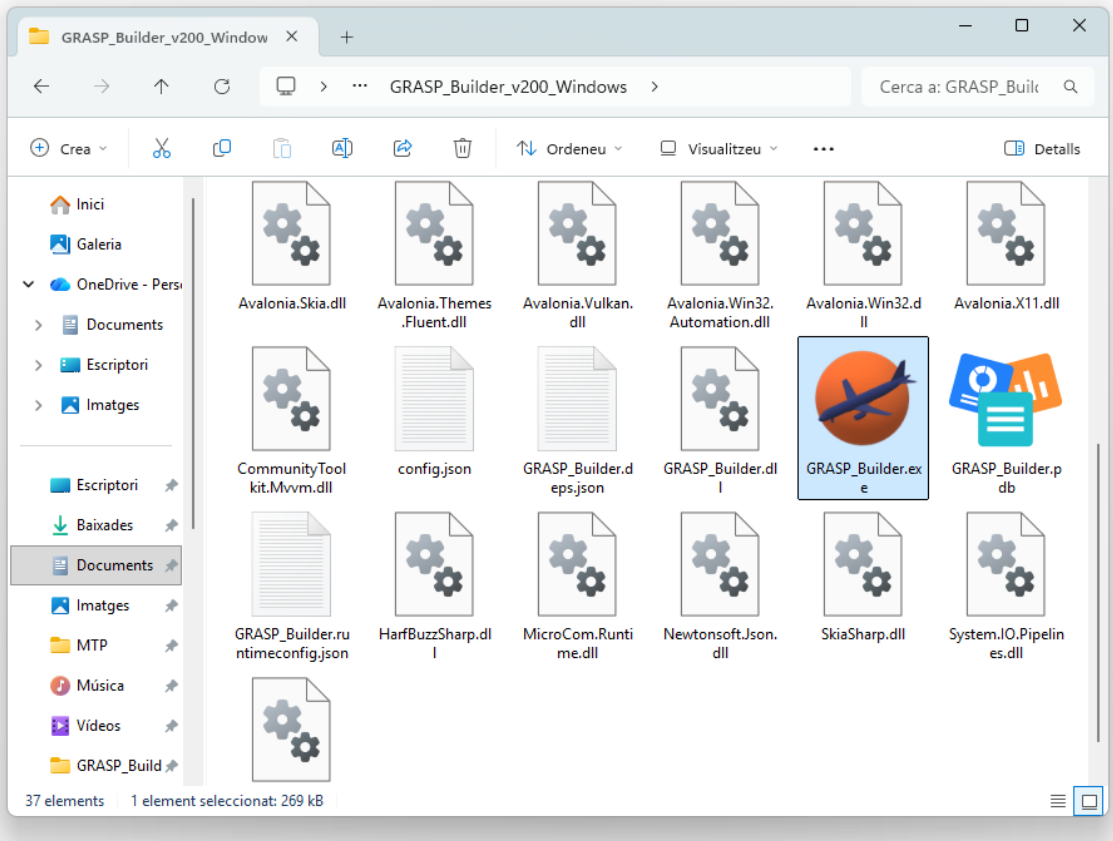


Figure 2: Executable in Windows

1.2 Linux: CALCULA

Similar procedure needs to be followed in the Linux enviroment, but it all can be executed via command line. In this case, it is needed an extra step, since Linux needs to have specified and application as executable.

In the desired folder, it is needed to open a command line. It can be done as it is shown in the following image:

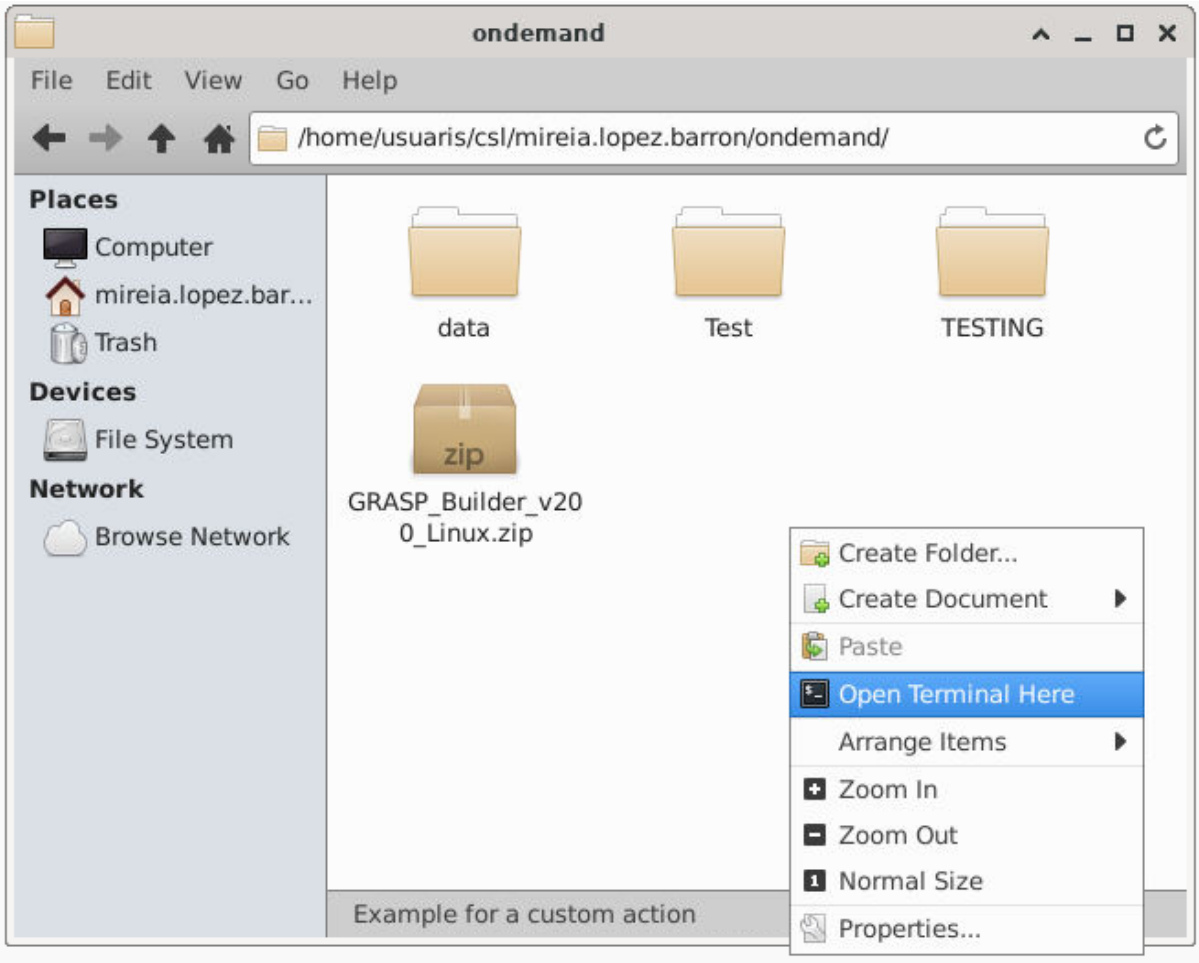


Figure 3: Open cmd in desired folder using Linux

Once it is opened, the following lines have to be executed, in order to unzip files, move to new folder and give executable permissions to the application.

```
1    unzip GRASP_Builder_v200_Linux.zip
2    cd GRASP_Builder_v200_Linux
3    chmod +x GRASP_Builder
```

With these commands, the application has been prepared to be opened whenever it is desired by using the command:

```
1    ./GRASP_Builder
```

Since this application is not shortcuted with a command, every time it is desired to be used, it has to be accessed the installation folder via command line and execute the calling command with the application name.

2 Project management

When the application is launched, the home screen is displayed showing three diferent buttons to manage projects.

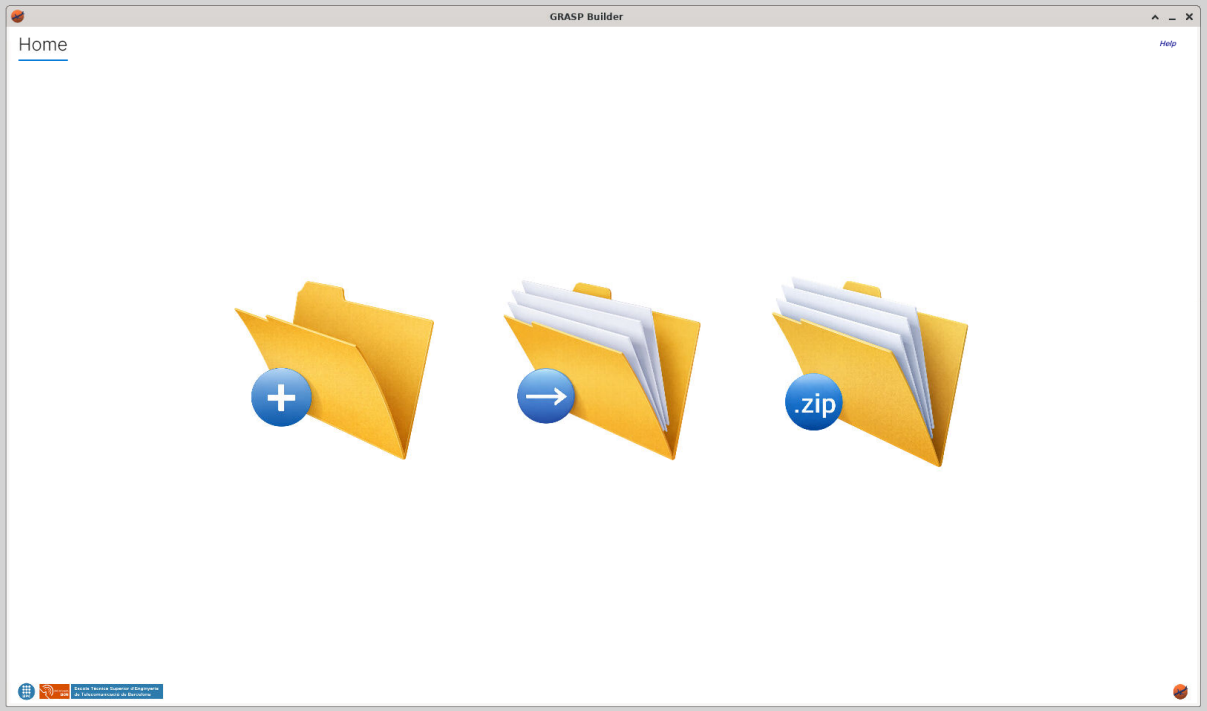


Figure 4: Home screen

First button will open a dialog to create a new project (Figure 5). Second button will open a dialog that allows the user to open an existing project (Figure 6). Third button will open a dialog that allows the user to import a project from a zip file (Figure 9).

When creating a new project, the user must provide a name for the project and select a location to save it. In the project folder, a new file called "project.grasp" will be created. This file will contain the corresponding project configuration.

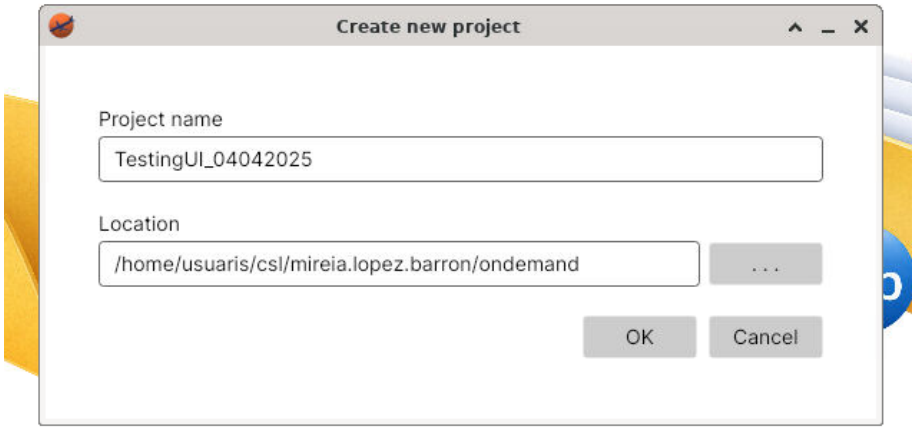


Figure 5: Create new project window

When opening a project, the user must select a project folder that contains an existing project.

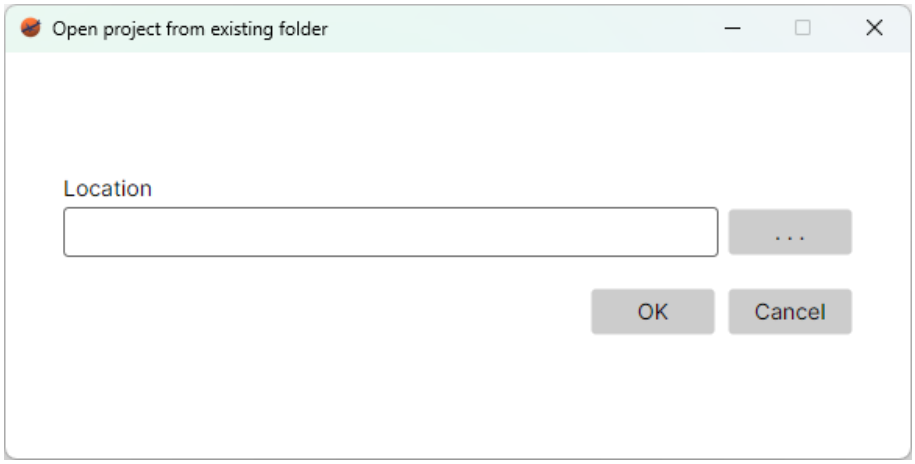


Figure 6: Open project window

All dialogs allow the user to navigate through the file system to select a project folder by using the browser button "..." (Figure 7).

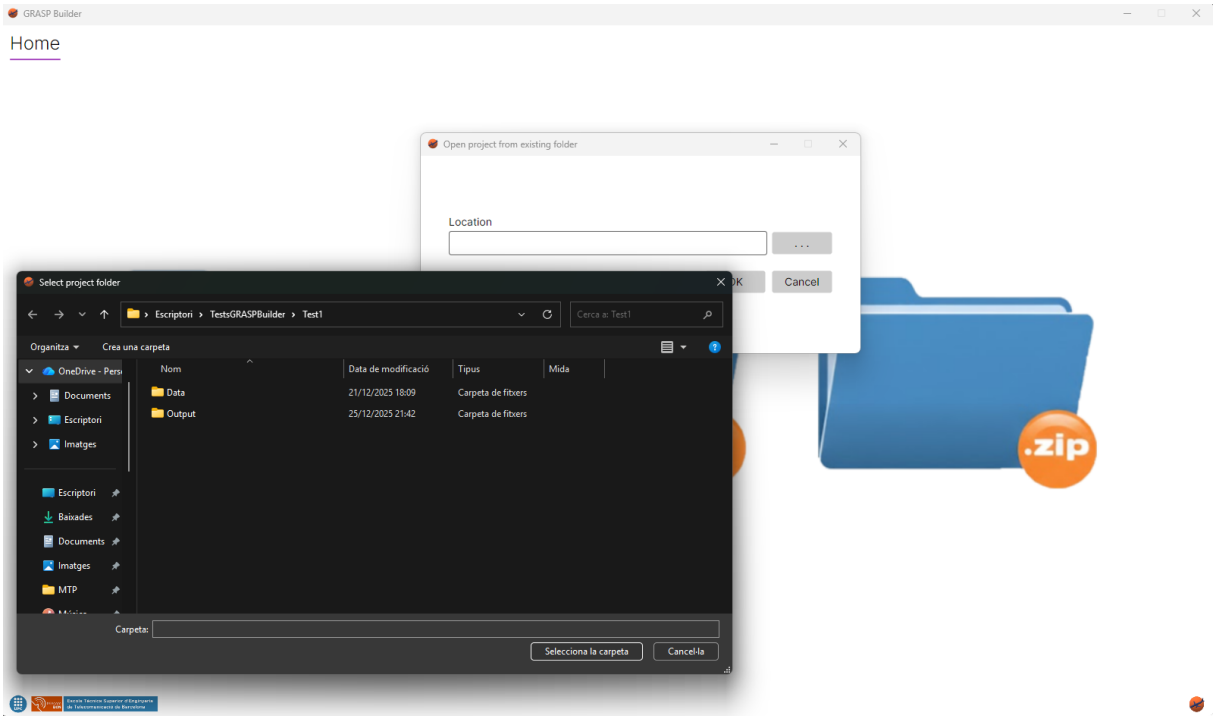


Figure 7: Open project browser

In the project folder, it is necessary it exists the file "project.grasp". Without this file, the application will not be able to load the project and an error message will be displayed (Figure 8).

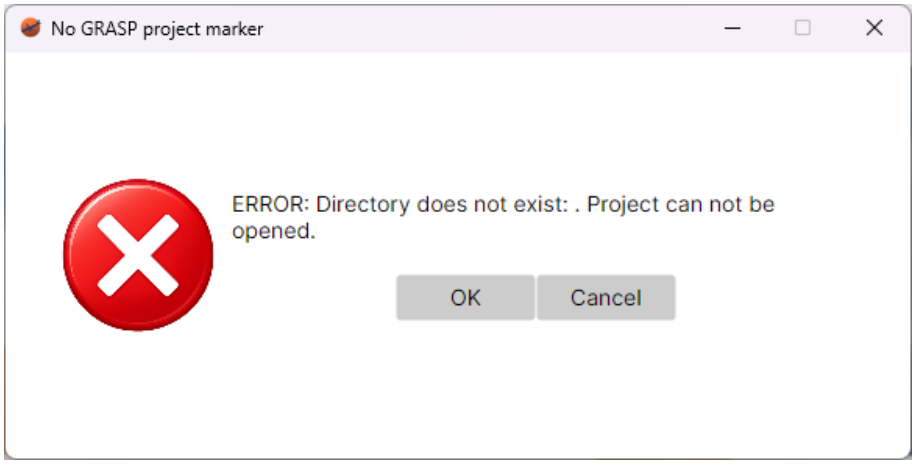


Figure 8: Error when opening a project

When importing a project, the user must select a zip file that contains an existing project. This zip file must also contain the file "project.grasp". Zip file should be located in the directory where the user wants the project to be created.

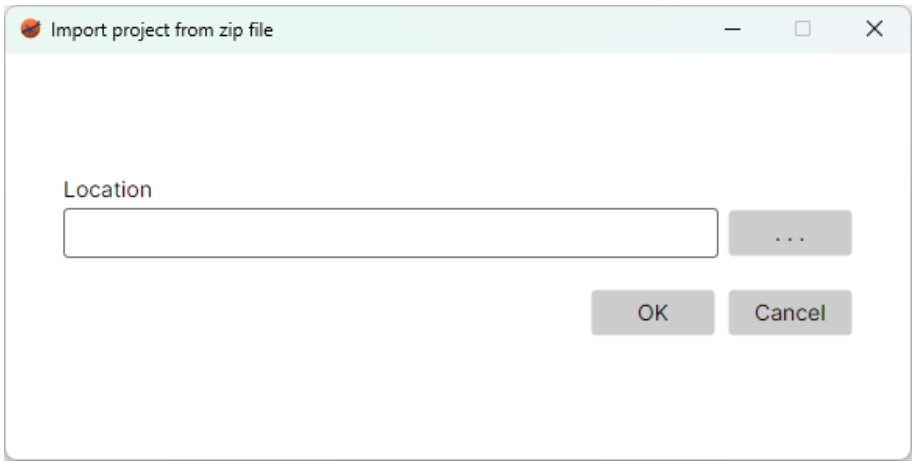


Figure 9: Import project window

Once a project is initialized, the user can access the File menu to perform different actions, in order to manage the current project (Figure 10).

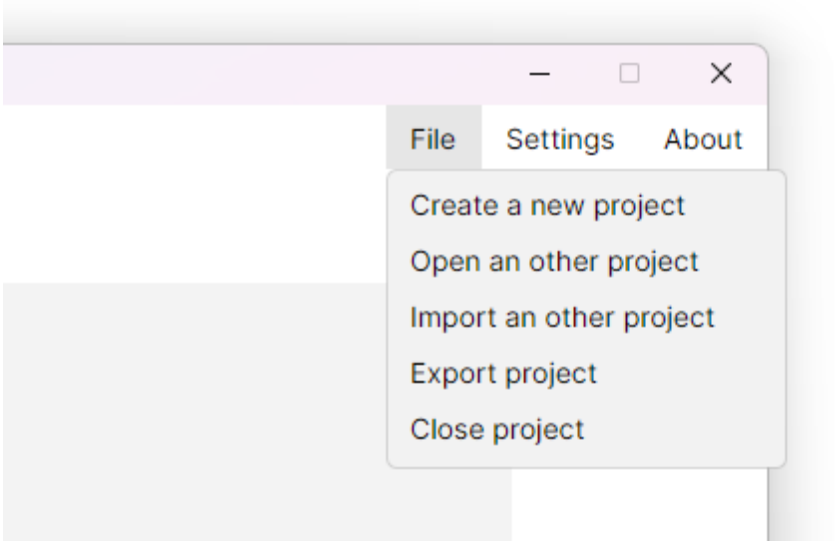


Figure 10: Menu options

Next to the File menu, it can be located the Settings button. This button opens a dialog that allows the user to configure different options of the application Figure 11.

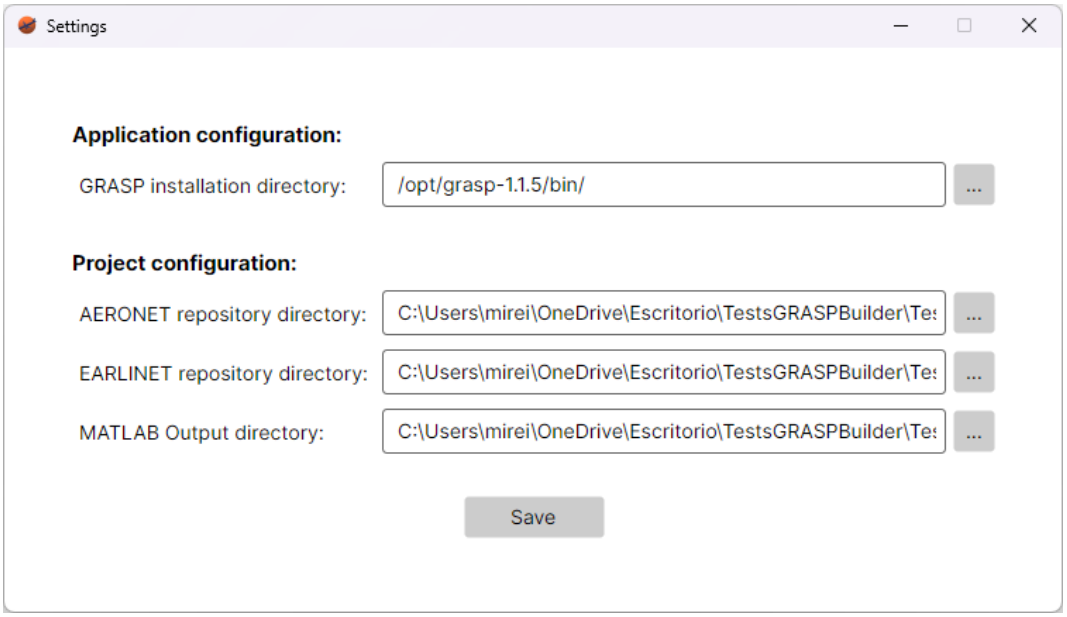


Figure 11: Settings window

The different parameters that can be configured are separated by application configuration parameters and project configuration parameters. The application configuration parameters have the same value for all projects. For now, it only includes the installation directory of GRASP algotithm.

Project configuration parameters are specific for each project. For now, it only includes the corresponding respository directories where the downloaded data will be stored for both, EARLIENT and AERONET sites, and the MATLAB output directory, where all the files created after running the corresponding scripts and GRASP algotithm will be created.

3 Download files

Once a project is created, the first step is to download the files from the different websites in order to have the data available.

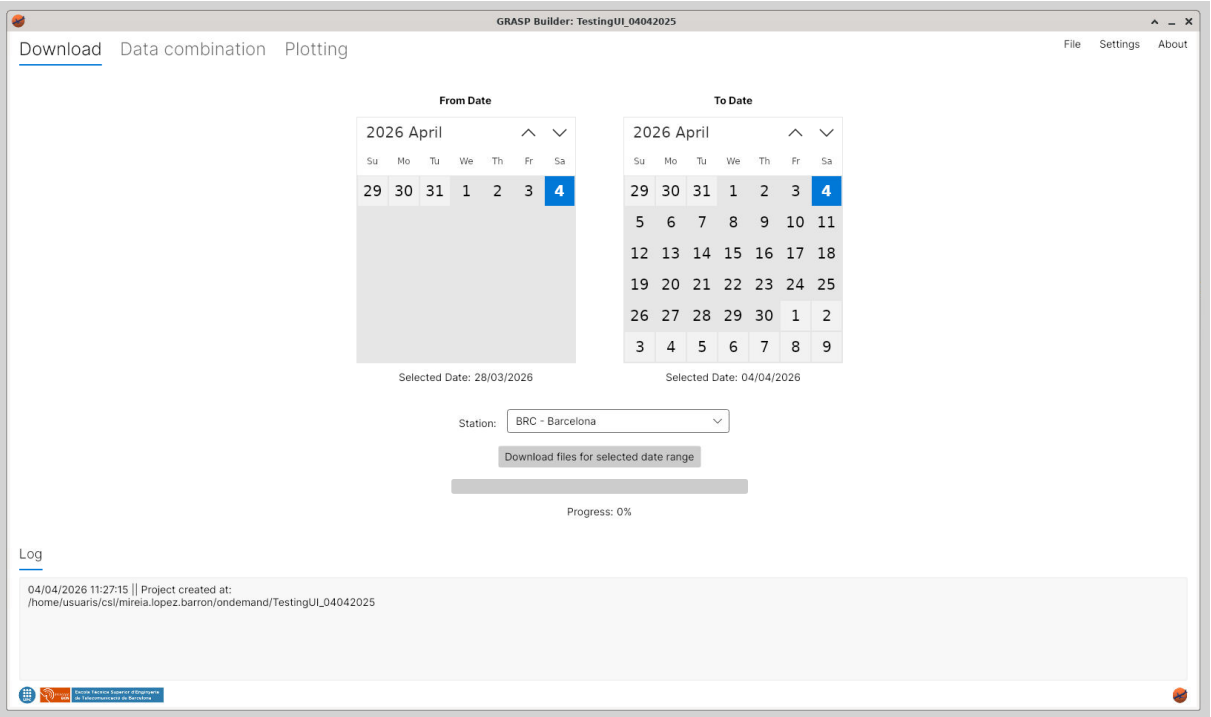


Figure 12: Project loaded

In this tab, the user is available to choose the range of dates to download the files from the different websites. For the moment, it is only available to download files from the selected site. Available sites are locations that have data in both websites, Aeronet and Earlinet. Sites that only have data in one of them are descarted, since it is necessary files from both platforms in order to run the GRASP algorithm.

During the process, all buttons will be disabled and the progress bar will show the percentage of files downloaded. Once all files are downloaded, a new section will be shown where the user will be able to see the list of files downloaded and select the files that desires to keep. This section will be also shown if the user opens a project with already downloaded files (Figure 13).

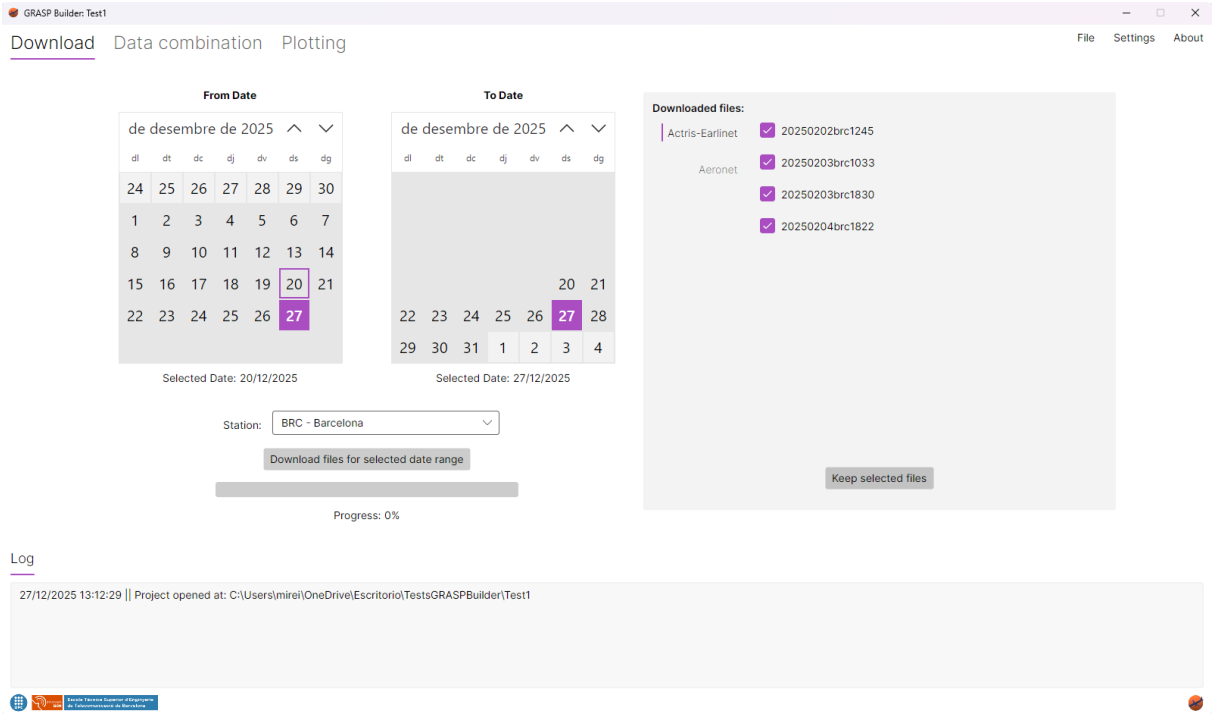


Figure 13: Files downloaded

4 Data Combination

Once all files have been downloaded, the user should the Data Combination tab. In this view, it can be selected a Measure ID from the options available (Figure 14). All measure IDs shown in downloaded files my be not available, since the list of measure IDs is filtered depending on the data contained. Measure IDs containing files with code 007 will not be added to the list, since these measurements will not contain all the data needed for the analysis.

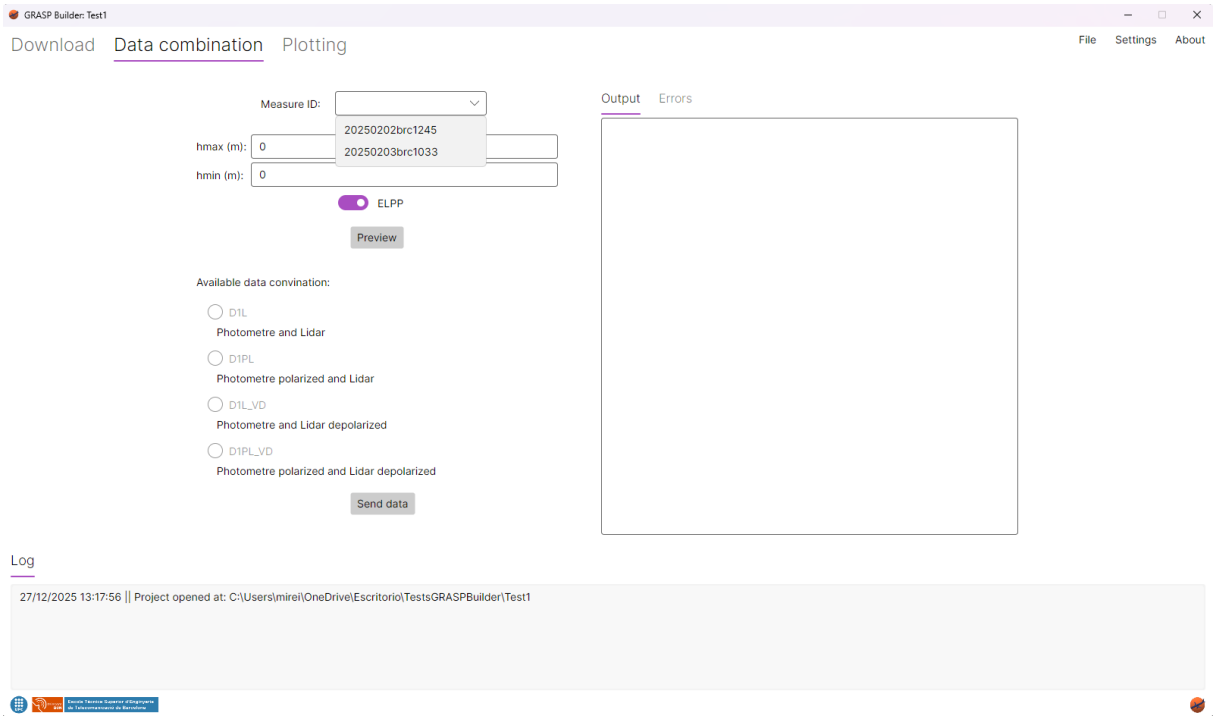


Figure 14: Data combination view

When a Measure ID is selected, the GetHeights script is executed, so the minimum and maximum height can be retrieved instead of having 0 values. The, the Preview script has to be launched in order to preview the data and obtain the available configuration options. This script can be launched manually using the "Preview" button all the desired times with the two different type of data (ELPP and VD, if possible). Default configured data type to show is ELPP, since this data is always available (Figure 15). The other option is VD (Figure 16).

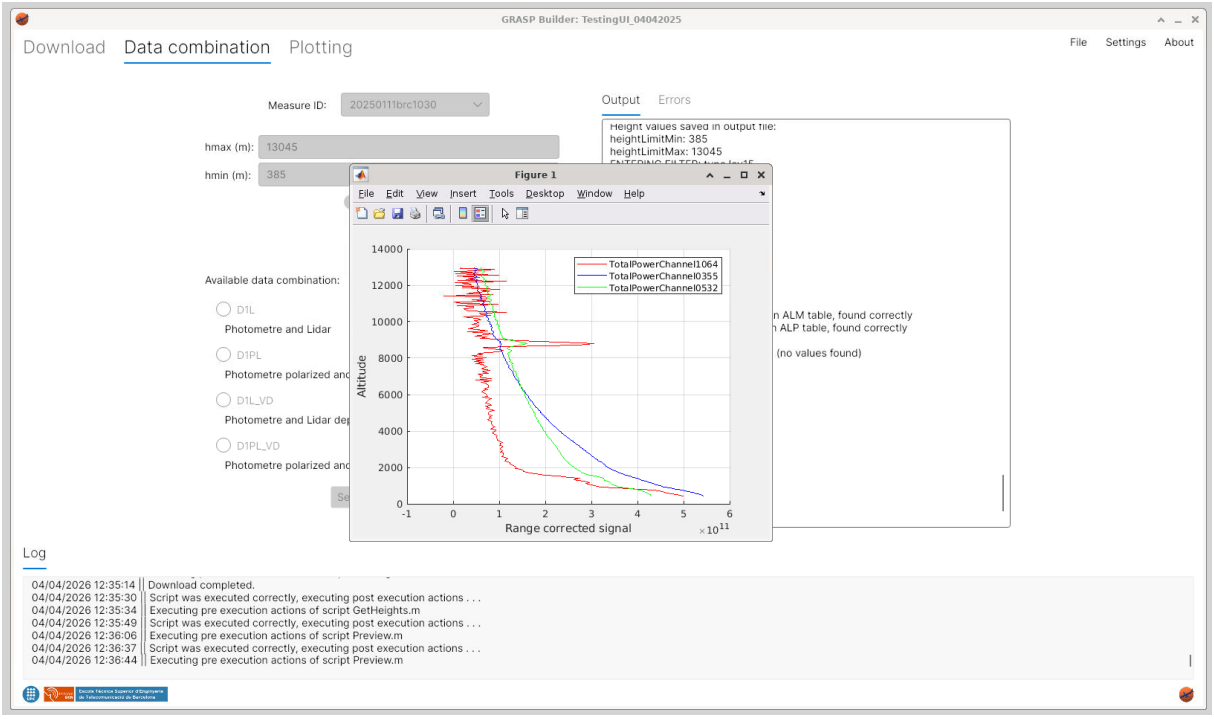


Figure 15: Execute preview with ELPP option

During the script execution, the previsualitzatin figure will be shown. In order to update the application with the available configurations, the Matalb figure needs to be closed, in order to execute the corresponding script post execution actions.

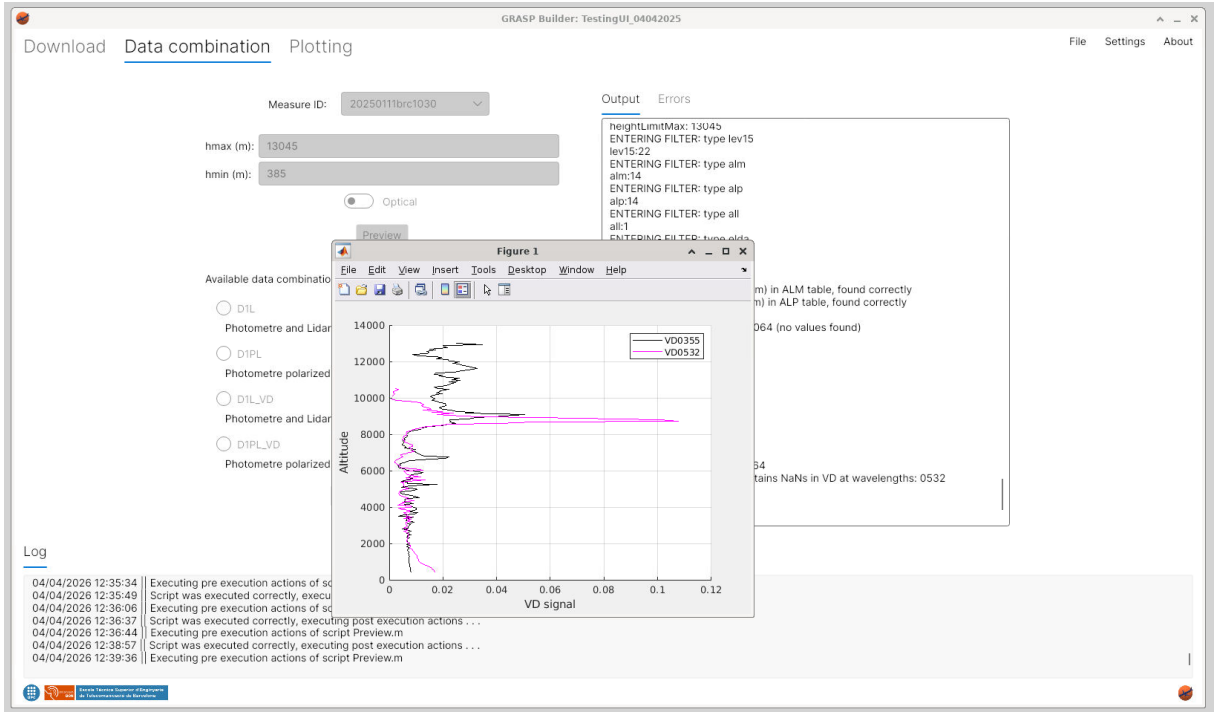


Figure 16: Execute preview with VD option and heighs defined

It is recommended to review the height values before starting the data combination process, since NaN values must be avoided in order to run the algorithm successfully.

Figure 17 shows and example of the preview window with the heights defined and Optical data selected.

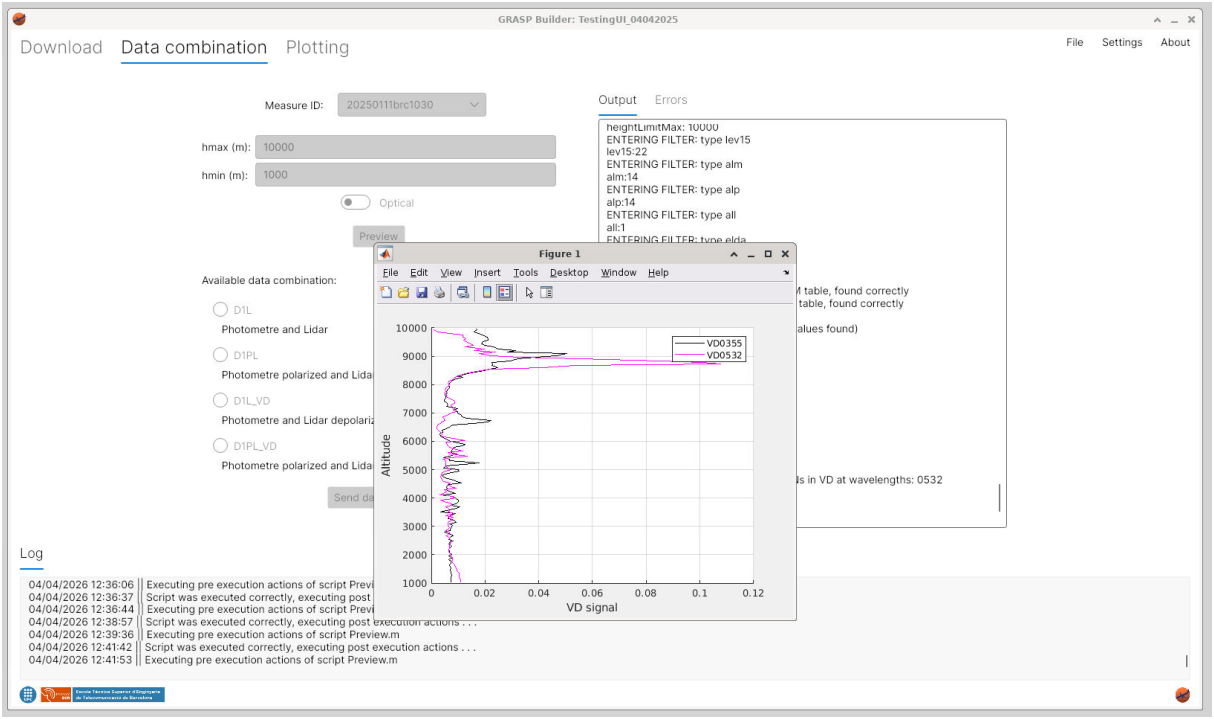


Figure 17: Execute preview with VD option

Every time a script in this window is executed, all buttons will be disabled, and once the plot window is closed, GUI elements will be updated and enabled again.

When a configuration is selected, by clicking the "Send ata" button, the SendFiles script is executed. If the script is executed successfully and .sdat file has been written successfully, .yml configuration file will be automatically created and GRASP algorithm will be executed.

Figure 18: Execute SendFiles script

Once GRASP has finished executing, it will appear an information message notifying the user Figure 18. If something went wrong, an error will appear.

5 Plotting

Once GRASP has been executed, the user can plot the result data in the Plotting tab (Figure 19).

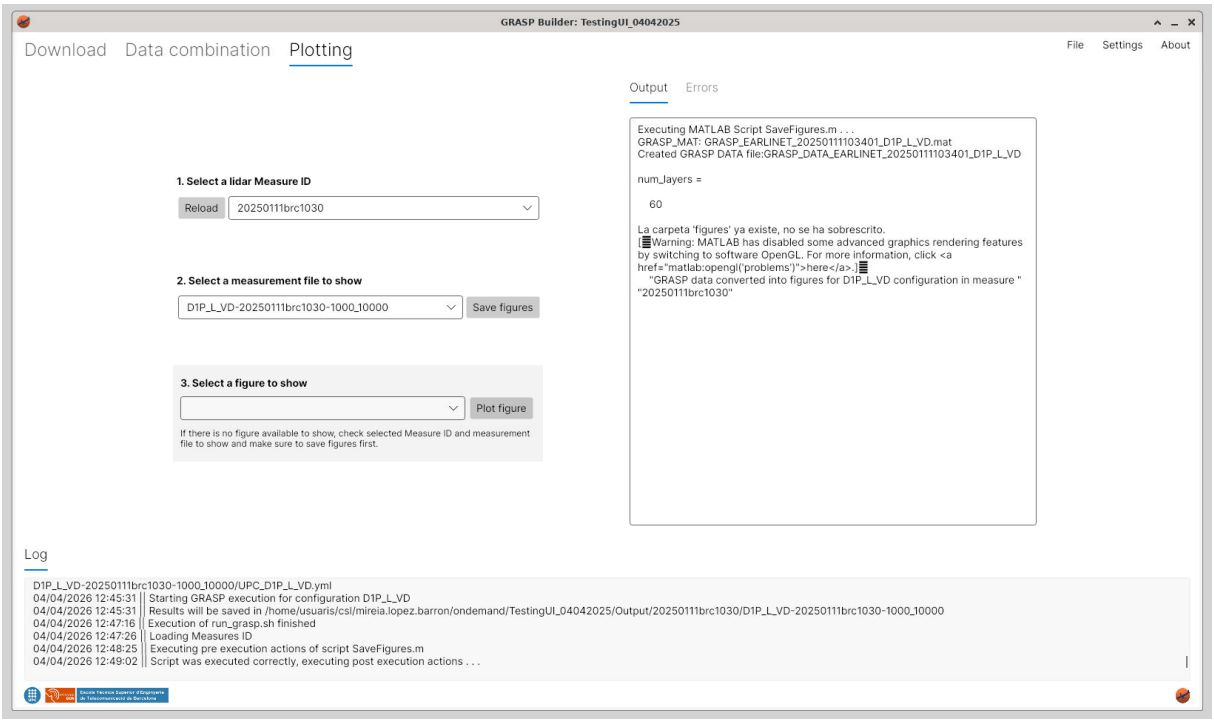


Figure 19: Plotting Tab with available Measure IDs

In this tab, the user can see the available Measure IDs and select the ones that desires to plot. If there are no Measure IDs available, click in "Reload" button, it will scan the MatalabOutputDirectory folder again.

Once the Measure ID is selected, the list of available measurement files will be updated automatically. There the user will be able to choose between the different options (Figure 20).

If this process has already been executed, it will not be needed to execute "Save figures". In the case it is not it is necessary to do it, in order to read all the GRASP output data and save it in the corresponding files.

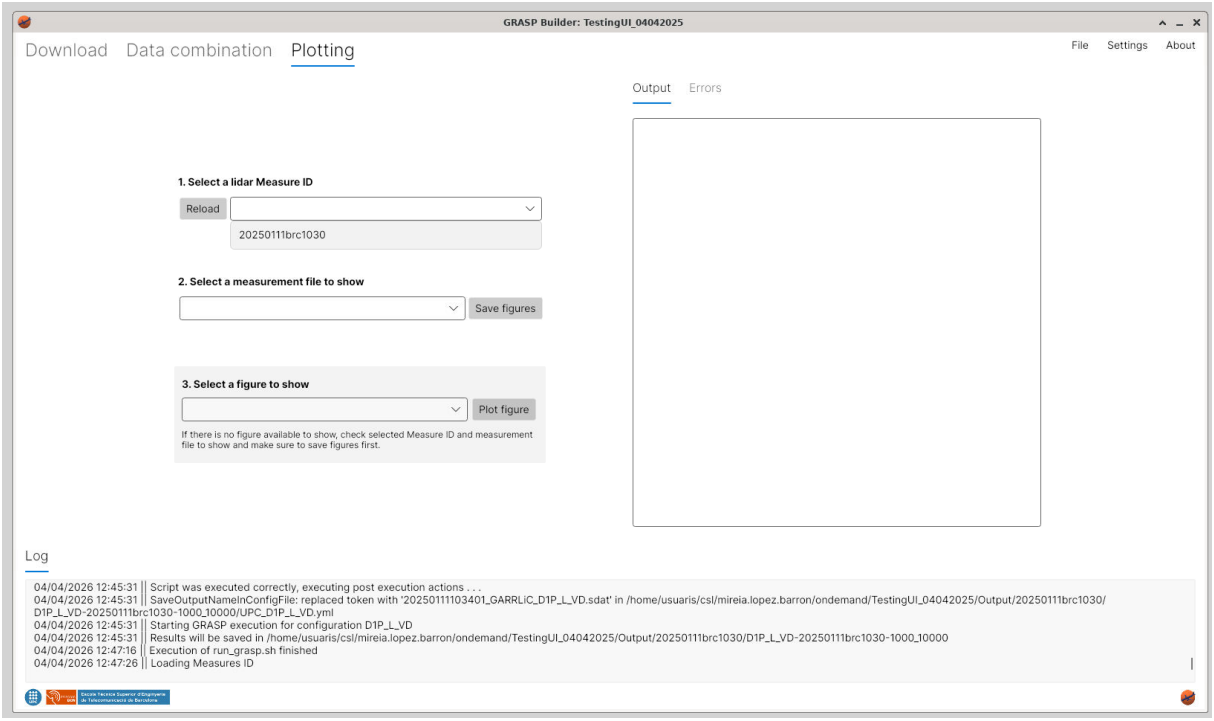


Figure 20: Plotting Tab with available measurement files

All available figures will be listed (Figure 21), the user will be able to choose between the different options.

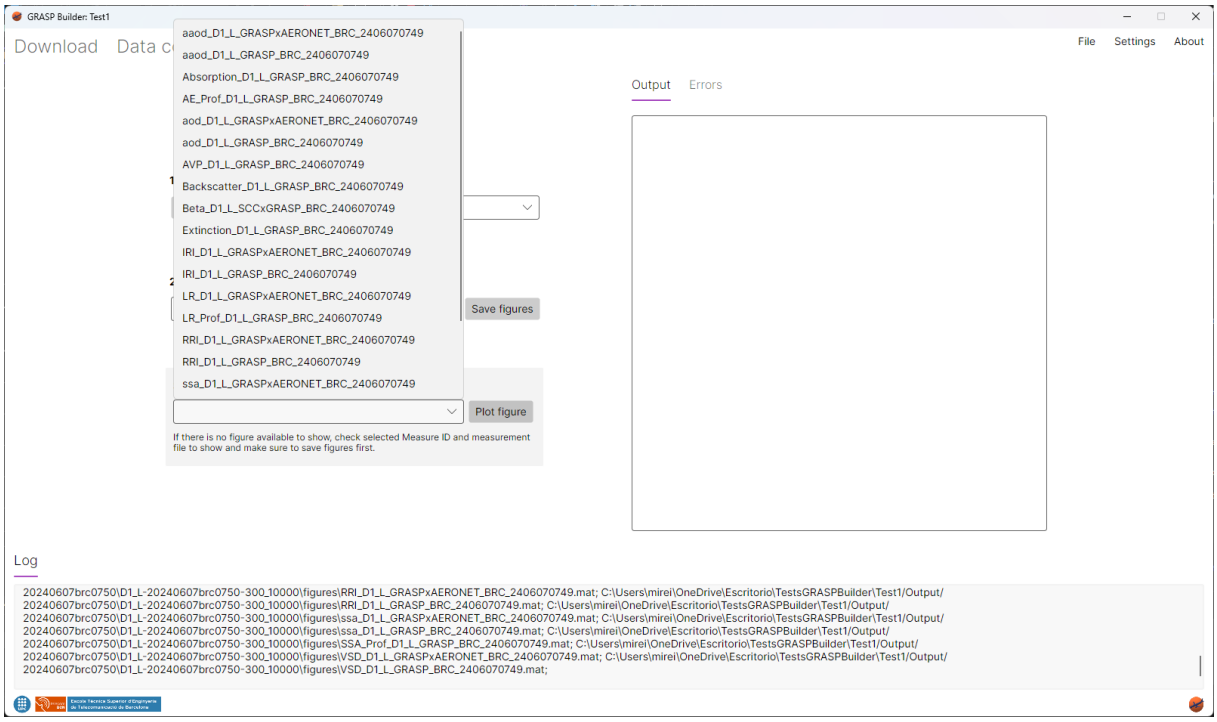


Figure 21: Plotting Tab with available figures

Once a figure is chosen, button "Plot figure" can be clicked in order to start corresponding MATLAB script execution. During the script execution, the corresponding figure will be shown (Figure 22).

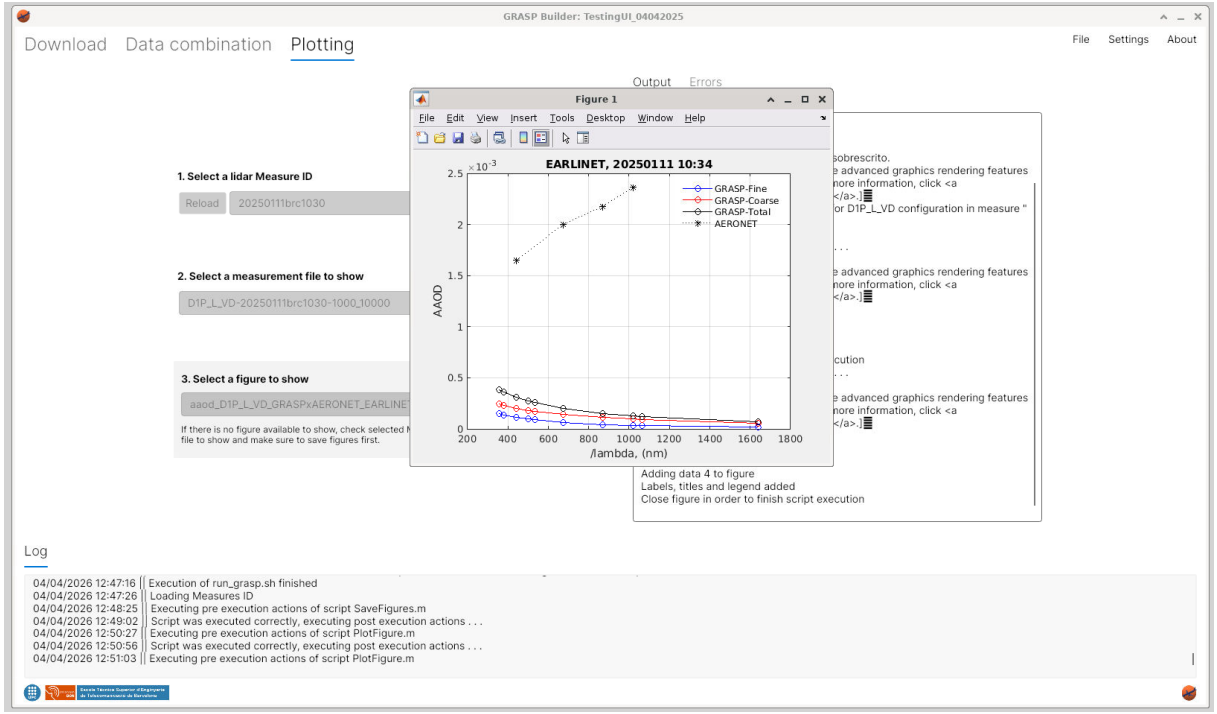


Figure 22: Plotted figure example